

ЛАБОРАТОРНАЯ РАБОТА «НАСЛЕДОВАНИЕ И ПОЛИМОРФИЗМ»

Цель работы:

- Ознакомиться с принципами объектно-ориентированного программирования: наследования и полиморфизм
- Изучить назначение и правила описания абстрактных классов и интерфейсов
- Получить представление о преимуществах использования полиморфизма при проектировании классов
- Научиться проектировать классы, используя механизм наследования
- Получить практические навыки создания иерархий наследования классов.

Порядок выполнения лабораторной работы

1. Создать иерархию, представленную UML-диаграммой, в соответствии с вариантом
2. Все классы (для которых это возможно) объявить конечными
3. Если UML-диаграмма некорректна или неоптимально, разрешено её подкорректировать (требуется обоснование)
4. По необходимости добавьте дополнительные поля в классы, для обеспечения возможности заявленных в описании функций.
5. Определить виды отношений между классами иерархии
6. Во всех классах реализовать специфичные методы, унаследованные от абстрактных классов и интерфейсов. Правило работы таких методов определить самостоятельно.
7. Для всех классов и интерфейсов, параметр типа List заменить на массив объектов (работа со списками предусмотрена в другой лабораторной работе)
8. Для защиты лабораторной работы достаточно наличие классов и интерфейсов представленной иерархии (т.е. демонстрация работы иерархии будет целью следующих лабораторных работ)
9. Подготовить ответы на контрольные вопросы
10. Сделать выводы о выполненной работе (включающие описание основных преимуществ использования наследования и полиморфизма)

Варианты заданий для самостоятельной работы

Вариант 1). Адвокатская контора

- Общая цель системы: поддержка внутренних бизнес-процессов адвокатской конторы — приём и ведение дел, распределение задач между сотрудниками, формирование отчётов и расчёт выплат (включая внешних консультантов).

- *Основные сущности и их роли:*

- Employee — абстрактный представитель сотрудника (общие данные: имя, идентификатор, должность, контакт).

- Lawyer — адвокат, реализует юридические услуги: принимает и ведёт дела (LegalCase), формирует отчёты по ведению дел, получает основную зарплату + бонусы за закрытые/успешно решённые дела.

- Assistant — помощник адвоката: выполняет вспомогательные задачи, готовит отчёты по задачам и получает фиксированный оклад.

- Consultant — внешний/внутренний консультант: предоставляет экспертную помощь почасово; для внешних — формируется плата по часам (computePayment), зарплата конторы может быть нулевой (IPayable применяется для расчёта выплат внешним).

- LegalCase — модель дела: идентификатор, клиент, описание, статус, сроки, ответственные, лог действий; метод toString() — текстовое представление дела.

- LawFirmManager — управляющий модуль: хранит список сотрудников, распределяет дела, агрегирует отчёты, запускает расчёт выплат.

- *Интерфейсы и обязанности:*

- ILegalService — контракт для работников, оказывающих юридические услуги (handleCase, getSpecialization).

- IReportable — контракт для объектов, умеющих генерировать отчёты (generateReport).

- IPayable — контракт для расчёта выплат внешним специалистам (computePayment / calculateSalary).

- *Ключевые взаимодействия и сценарии:*

1. Приём дела: менеджер регистрирует LegalCase, определяет специализацию и назначает ответственного Lawyer (или команду).

2. Ведение дела: Lawyer через handleCase обновляет статус, логирует этапы; Assistant помогает в задачах и собирает материалы.

3. Отчётность: по требованию или по расписанию сотрудники реализующие IReportable генерируют отчёты; менеджер агрегирует их для клиента или внутренней бухгалтерии.

4. Выплаты: IPayable используется для расчёта оплаты внешним консультантам (почасовая оплата), а также для вычисления итоговой зарплаты сотрудников (calculateSalary учитывает базу и бонусы).

- *Ограничения и важные требования:*

- Сохранение истории действий по делу (аудит).

- Учет многопользовательской работы над одним делом (множественные участники).

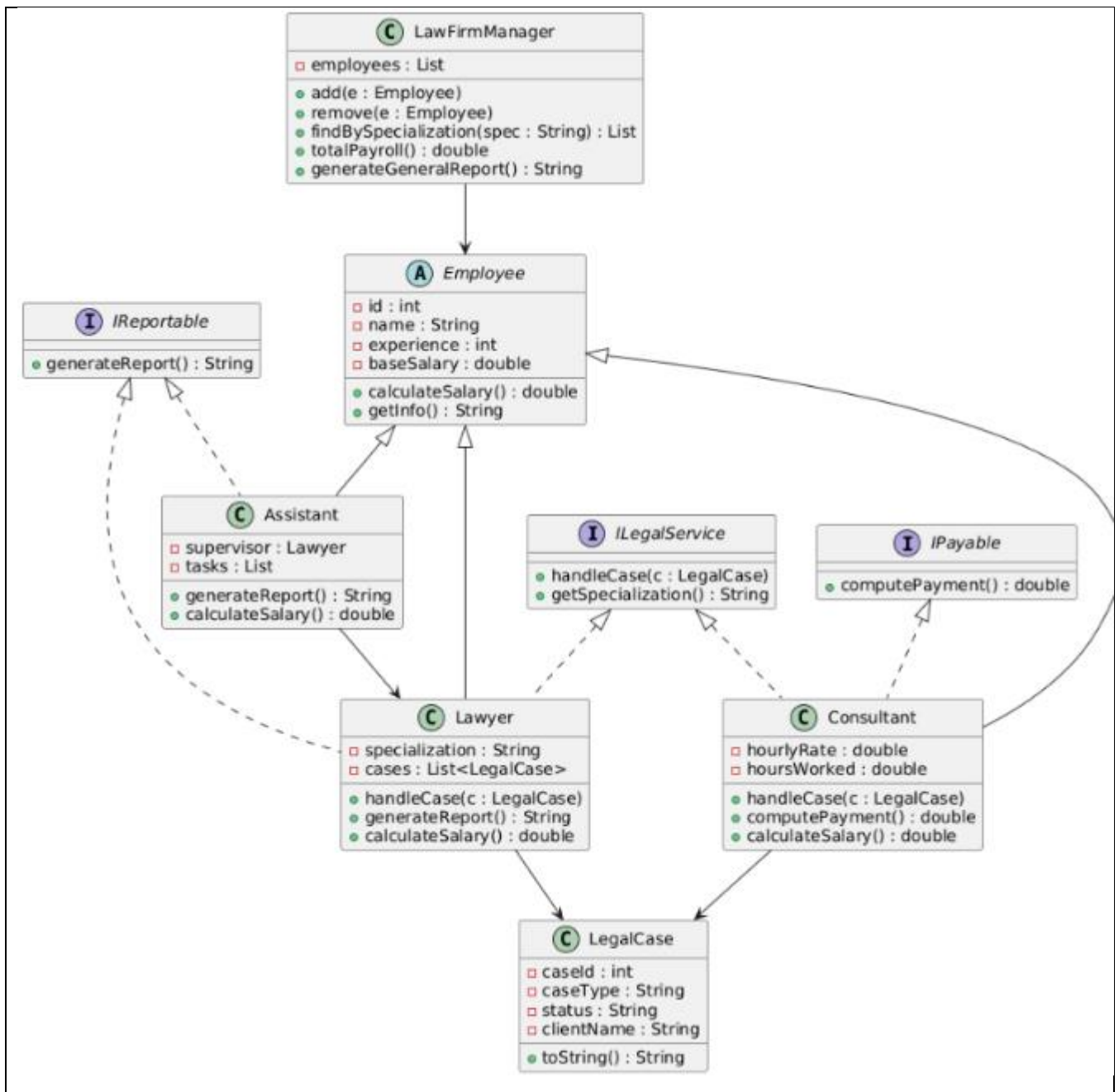
- Поддержка разных типов оплаты (фикс./почасово/бонусы).

- Защита персональных и клиентских данных (конфиденциальность).

- *Короткие варианты использования (Use Cases):*

- Регистрация нового дела → назначение адвоката → выполнение шагов дела → формирование отчёта → расчёт выплат.

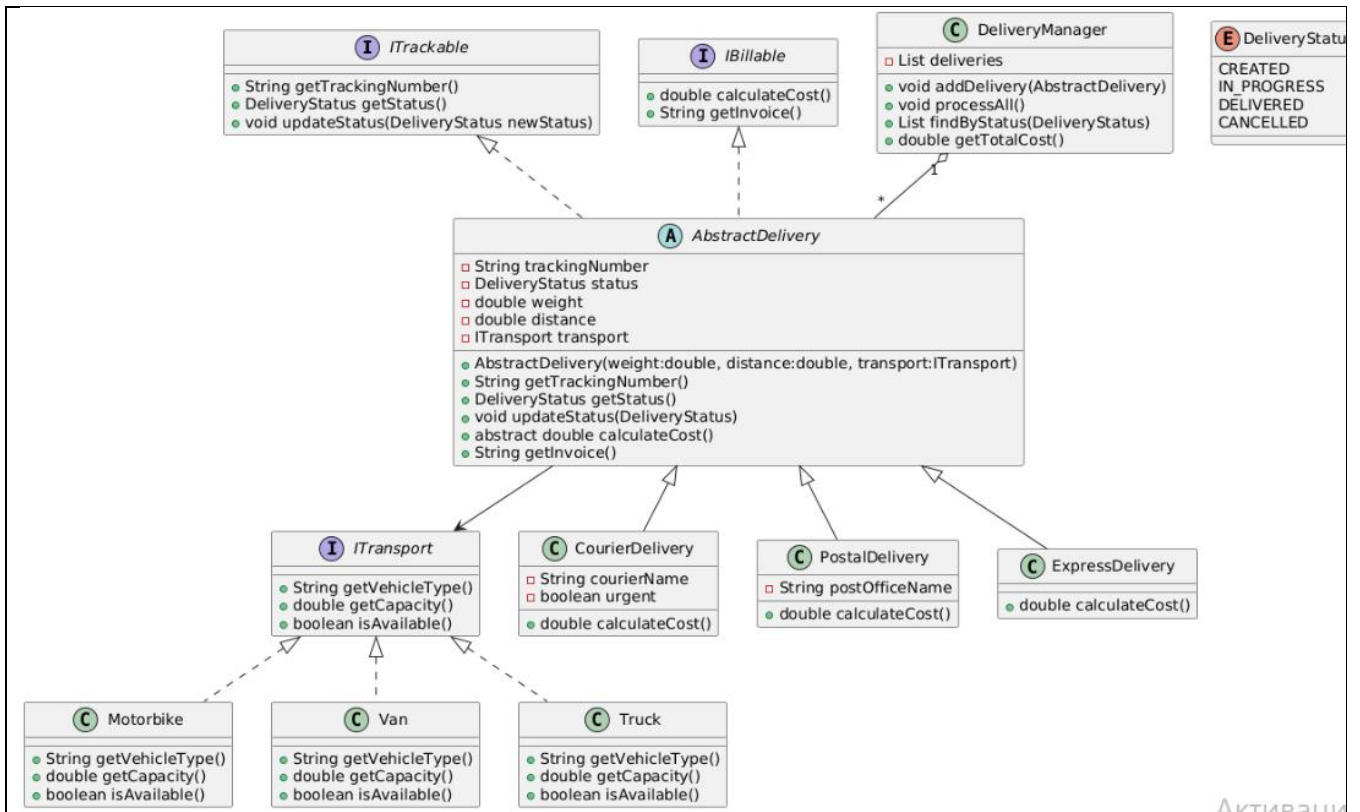
- Назначение внешнего консультанта к делу → учёт часов работы → расчёт и выплата консультационных услуг.



Вариант 2). Система управления доставкой товара.

Общая цель системы: Автоматизация и оптимизация процессов управления доставкой товаров, включая регистрацию, расчет стоимости, отслеживание статусов и выбор транспорта.

- Основные сущности и роли:
 - DeliveryManager: Управляет списком доставок, выполняет обработку, поиск и расчет стоимости.
 - AbstractDelivery: Базовый класс для всех типов доставок, содержит общую логику и абстрактный метод для расчета стоимости.
 - CourierDelivery, PostalDelivery, ExpressDelivery: Конкретные типы доставок с различной логикой расчета стоимости и используемым транспортом.
 - ITransport (Motorbike, Van, Truck): Определяют средства транспортировки для доставок.
 - DeliveryStatus: Enum, определяющий возможные статусы доставки (CREATED, IN_PROGRESS, DELIVERED, CANCELLED).
- Интерфейсы и обязанности:
 - ITrackable: Позволяет отслеживать состояние доставки. Обязанность: Предоставлять информацию о текущем статусе и местоположении.
 - IBillable: Позволяет рассчитать стоимость доставки и выставить счет. Обязанность: Предоставлять механизм для расчета стоимости доставки.
 - ITransport: Определяет характеристики средства транспортировки. Обязанность: Обеспечивать информацию о доступных транспортных средствах и их пригодности для конкретной доставки.
- Ключевые взаимодействия и сценарии:
 - Добавление доставки: DeliveryManager добавляет новый экземпляр AbstractDelivery (одного из конкретных типов) в список доставок.
 - Обработка доставки: DeliveryManager инициирует обработку всех доставок в списке. Каждый AbstractDelivery рассчитывает свою стоимость и может обновлять статус.
 - Расчет стоимости: Каждый AbstractDelivery (CourierDelivery, PostalDelivery, ExpressDelivery) реализует свой алгоритм расчета стоимости на основе различных факторов.
 - Отслеживание статуса: Каждая доставка реализует интерфейс ITrackable и может менять свой статус.
 - Поиск доставок: DeliveryManager предоставляет возможность поиска доставок по определенному DeliveryStatus.
 - Использование транспорта: Каждый тип доставки (CourierDelivery, ExpressDelivery) привязан к определенному типу транспорта (Motorbike, Van) реализующему ITransport.
- Ограничения и важные требования:
 - Система должна корректно рассчитывать стоимость различных типов доставок.
 - Система должна обеспечивать возможность отслеживания статуса доставки.
 - Система должна эффективно управлять большим количеством доставок.
 - Гибкость для добавления новых типов доставок и транспортных средств.
- Короткие варианты использования:
 - Добавить новую курьерскую доставку.
 - Рассчитать стоимость почтовой доставки.
 - Изменить статус доставки на "В процессе".
 - Найти все доставки со статусом "Доставлено".
 - Получить общую стоимость всех доставок.



Вариант 3). Программа телепередач.

Общая цель системы: Предоставление структурированной информации о телепрограмме, позволяющей управлять списком передач, отображать их свойства, оценивать и записывать их.

Основные сущности и роли:

- ProgramItem: Абстрактное представление любой телепередачи (новостная программа, фильм, спортивное событие, эпизод сериала).
- NewsProgram, Movie, SportsEvent, SeriesEpisode: Конкретные типы телепередач с уникальными характеристиками.
- ProgramSchedule: Менеджер списка передач, обеспечивающий добавление, удаление, отображение и сортировку.
- IBroadcastable: Интерфейс, определяющий возможность "показа" передачи.
- IRated: Интерфейс, определяющий наличие рейтинга у передачи.
- IRecordable: Интерфейс, определяющий возможность записи передачи.

Интерфейсы и обязанности:

- IBroadcastable: Обеспечивает метод Broadcast() (показать передачу) и GetDuration() (получить длительность).
- IRated: Предоставляет свойство Rating (рейтинг) и метод UpdateRating() (обновить рейтинг).
- IRecordable: Определяет метод Record() (записать передачу).
- ProgramItem: Определяет общие свойства всех передач и предоставляет метод ShowInfo() (отобразить информацию).
- ProgramSchedule: Управляет списком ProgramItem, обеспечивает методы добавления, удаления, отображения, сортировки и расчета среднего рейтинга.

Ключевые взаимодействия и сценарии:

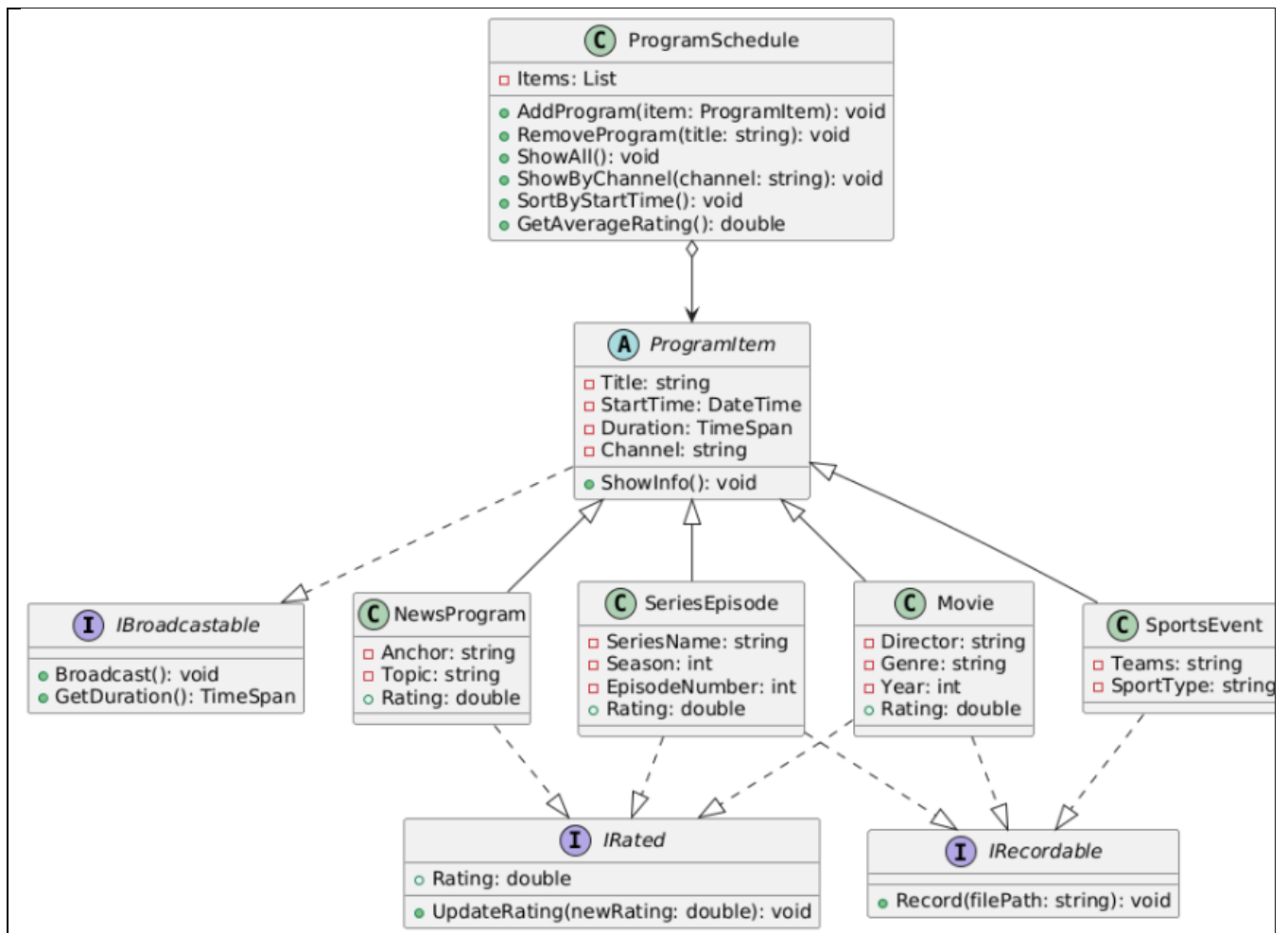
- Добавление передачи: ProgramSchedule.AddProgram() добавляет экземпляр одного из конкретных классов (NewsProgram, Movie, etc.) в список передач.
- Просмотр расписания: ProgramSchedule.ShowAll() или ProgramSchedule.ShowByChannel() отображает информацию о передачах, вызывая ProgramItem.ShowInfo() для каждой передачи.
- Запись фильма: Movie.Record() записывает фильм в указанный файл.
- Оценка новостной программы: NewsProgram.UpdateRating() обновляет рейтинг программы.
- Получение длительности передачи: Вызывается метод GetDuration() для экземпляра ProgramItem для определения длительности показа.

Ограничения и важные требования:

- SportsEvent не имеет рейтинга (не реализует IRated).
- Не все передачи можно записать (реализуют IRecordable выборочно).
- ProgramItem является абстрактным классом, экземпляры которого нельзя создавать напрямую. Необходимо использовать конкретные классы.

Короткие варианты использования:

- Добавить фильм "Интерстеллар" в расписание.
- Показать все передачи на канале "Культура".
- Отсортировать расписание по времени начала передач.
- Записать эпизод сериала "Игра престолов".
- Узнать средний рейтинг всех передач в расписании.



Вариант 4). Гостиница.

Общая цель системы: Управление номерным фондом гостиницы, резервирование номеров и предоставление услуг гостям.

Основные сущности и роли:

- Room: Абстрактное представление номера в гостинице.
- SingleRoom, DoubleRoom, Suite: Конкретные типы номеров, отличающиеся характеристиками.
- Hotel: Управляет списком номеров и предоставляет услуги.
- IReservable: Интерфейс для резервирования и отмены резервирования номеров.
- IService: Интерфейс для предоставления гостиничных услуг.

Интерфейсы и обязанности:

- IReservable: Определяет методы Reserve() (зарезервировать номер) и CancelReservation() (отменить резервирование).
- IService: Определяет метод ProvideService() (предоставить услугу).
- Room: Предоставляет общие свойства номеров (номер, вместимость, цена) и метод GetRoomInfo() (получить информацию о номере). Метод CheckAvailability() абстрактный и должен быть реализован в дочерних классах.
- Hotel: Управляет списком номеров, предоставляет методы AddRoom() (добавить номер), GetAvailableRooms() (получить доступные номера) и ProvideService() (предоставить услугу).

Ключевые взаимодействия и сценарии:

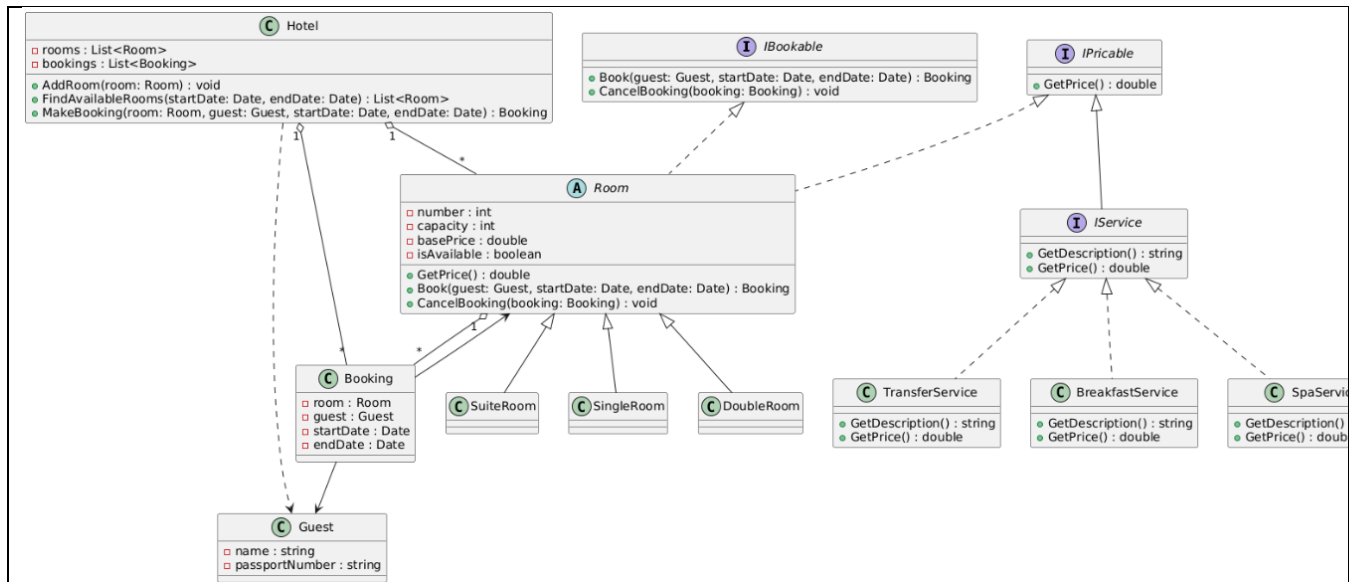
- Добавление номера: Hotel.AddRoom() добавляет экземпляр SingleRoom, DoubleRoom или Suite в список номеров.
- Получение списка доступных номеров: Hotel.GetAvailableRooms() возвращает список номеров, для которых CheckAvailability() возвращает true.
- Резервирование номера: Room.Reserve() (через интерфейс IReservable) резервирует номер, при условии его доступности.
- Предоставление услуги: Hotel.ProvideService() предоставляет запрошенную услугу (реализация зависит от конкретной услуги, реализующей IService).

Ограничения и важные требования:

- Метод CheckAvailability() должен учитывать текущие резервации.
- Room - абстрактный класс, напрямую создать объект Room нельзя.
- Предоставление сервисов делегируется классу Hotel, но реализация предоставления сервиса лежит вне этой модели. Например, необходим класс CleaningService реализующий IService.
- Реализация Reserve() в классах комнат должна учитывать логику резервации (например, обновление статуса доступности комнаты).

Короткие варианты использования:

- Добавить двухместный номер в гостиницу.
- Получить список доступных номеров на определенную дату.
- Забронировать одноместный номер.
- Отменить бронирование номера.
- Заказать уборку в номере.



Вариант 5). Реализация готовой продукции.

Общая цель системы:

Управление товарным ассортиментом, отслеживание сроков годности, предоставление скидок, организация доставки и ведение учета продаж.

Основные сущности и роли:

- Product: Общее представление товара, содержащее базовую информацию (название, цена, количество, дата производства).
- FoodProduct: Продовольственный товар, имеющий срок годности и возможность списания.
- ClothingProduct: Одежда, имеющая размер, цвет и возможность доставки.
- ElectronicsProduct: Электроника, имеющая гарантийный срок и возможность доставки.
- ProductList: Каталог товаров, обеспечивающий добавление, удаление, поиск и фильтрацию товаров.
- Система: Обеспечивает взаимодействие между сущностями и реализацию бизнес-логики.

Интерфейсы и обязанности:

- IWriteOffable: Обеспечивает возможность списания товара (истек срок годности).
- IDeliverable: Обеспечивает возможность доставки товара по адресу.
- IDiscountable: Обеспечивает возможность применения скидки к товару.

Ключевые взаимодействия и сценарии:

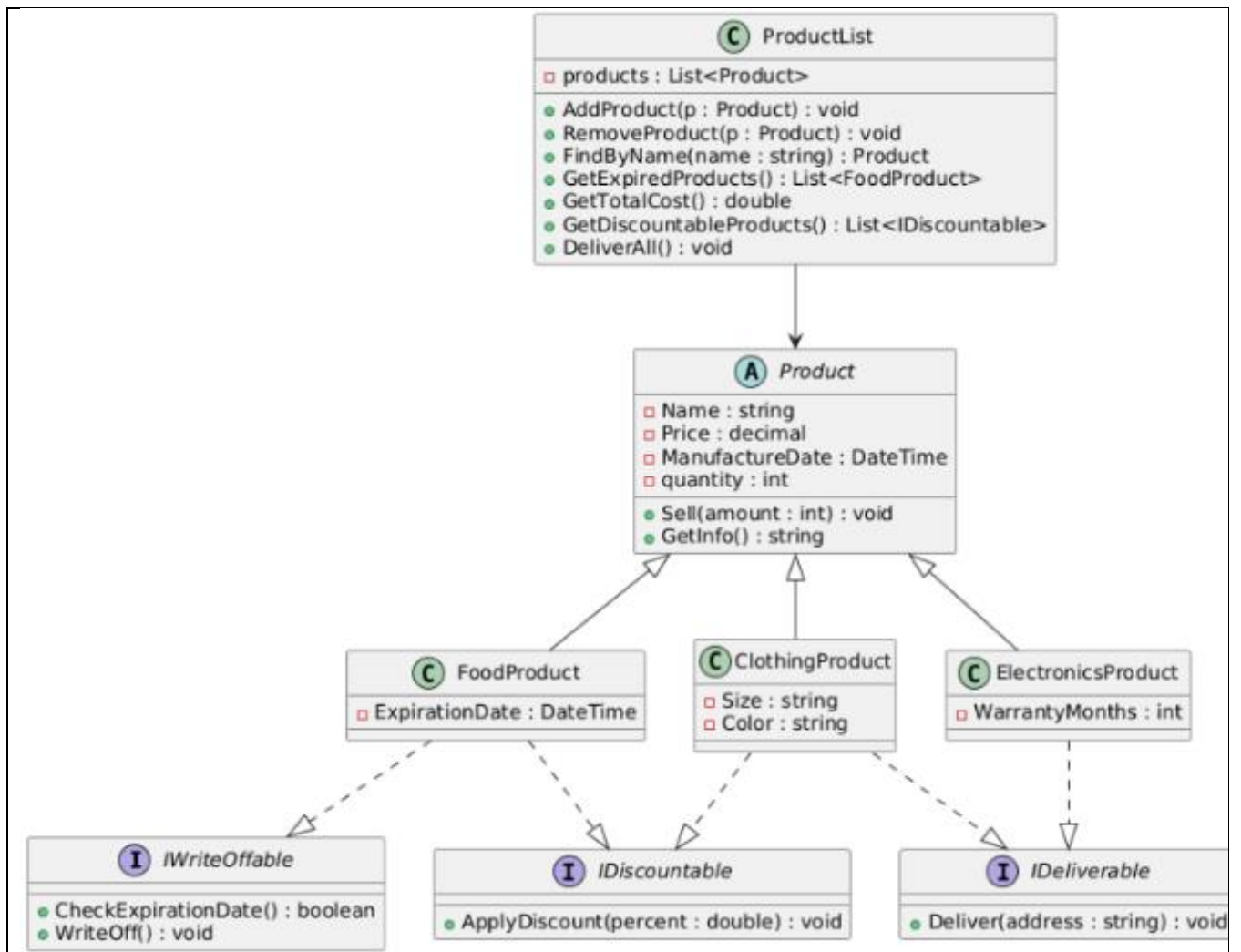
- Добавление товара: Система добавляет новый товар в ProductList.
- Продажа товара: Система уменьшает количество товара в ProductList на указанное значение.
- Проверка срока годности: Система проверяет срок годности FoodProduct и списывает его при необходимости.
- Применение скидки: Система применяет скидку к FoodProduct или ClothingProduct.
- Доставка товара: Система организует доставку ClothingProduct или ElectronicsProduct по указанному адресу.
- Поиск товара: Система ищет товар в ProductList по названию.
- Получение просроченных товаров: Система возвращает список просроченных FoodProduct из ProductList.
- Расчет общей стоимости: Система вычисляет общую стоимость всех товаров в ProductList.
- Получение товаров со скидкой: Система возвращает список товаров, к которым можно применить скидку.
- Организация доставки всех товаров: Система организует доставку для всех доставляемых товаров в списке.

Ограничения и важные требования:

- Скидки применяются только к FoodProduct и ClothingProduct.
- Доставка осуществляется только для ClothingProduct и ElectronicsProduct.
- Списание товара возможно только для FoodProduct.
- Проверка сроков годности осуществляется только для FoodProduct.

Короткие варианты использования:

- Добавить новый товар "Яблоки" в каталог.
- Продать 5 единиц товара "Футболка".
- Проверить и списать просроченные продукты.
- Применить скидку 10% на товар "Хлеб".
- Организовать доставку товара "Телевизор" по адресу "ул. Ленина, 1".
- Найти товар "Молоко" в каталоге.
- Получить список всех товаров, на которые можно применить скидку.
- Рассчитать общую стоимость всех товаров в каталоге.



Вариант 6). Учет успеваемости студентов ВУЗА.

Общая цель системы: Автоматизация процессов учёта успеваемости студентов, предоставление данных для анализа и формирования отчётности.

Основные сущности и роли:

- Студент (Student): Основной объект учета, характеризуется личными данными, идентификатором и списком оценок.
- Преподаватель (Teacher): Лицо, ведущее учебные курсы и имеющее право формировать отчетность об успеваемости.
- Курс (Course): Учебный предмет, содержащий информацию о названии, преподавателе и списке студентов.
- Оценка (Grade): Информация об успеваемости студента по конкретному предмету в определенную дату.
- Группа (Group): Объединение студентов для организации учебного процесса.

Интерфейсы и обязанности:

- IAssessable: Определяет возможность добавления оценок и расчета среднего балла для объектов (Student).
- IReportable: Определяет возможность формирования отчета об успеваемости для объектов (Student, Teacher).

Ключевые взаимодействия и сценарии:

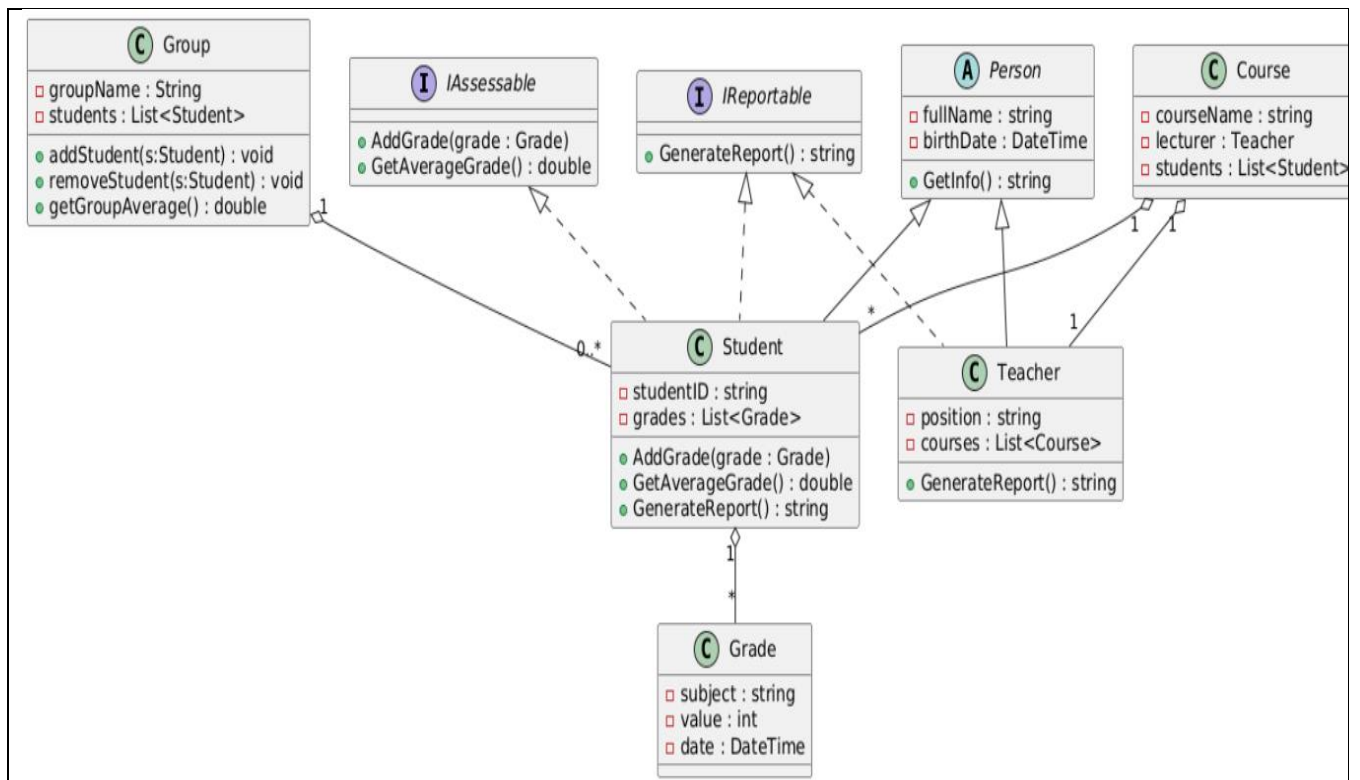
- Преподаватель добавляет оценки студентам за конкретный курс.
- Система рассчитывает средний балл студента по всем предметам.
- Преподаватель формирует отчет об успеваемости группы студентов.
- Студент просматривает свой список оценок и средний балл.
- Система предоставляет данные для анализа успеваемости студентов по различным критериям.

Ограничения и важные требования:

- Необходимо обеспечить корректность и целостность данных об успеваемости.
- Требуется возможность формирования отчетов в различных форматах.
- Система должна обеспечивать разграничение прав доступа для различных ролей (студент, преподаватель).

Короткие варианты использования:

- Добавление оценки студенту: Преподаватель вводит оценку для конкретного студента по определенному предмету.
- Просмотр успеваемости студента: Студент просматривает список своих оценок и средний балл.
- Формирование отчёта по группе: Преподаватель генерирует отчёт об успеваемости студентов в группе.
- Расчет среднего балла группы: Система вычисляет средний балл для группы студентов.



Вариант 7). Автоматизация работы факультета.

Общая цель системы: Управление и автоматизация основных процессов факультета, включая управление персоналом (студенты, преподаватели, сотрудники), учебными курсами и зачислением/распределением по курсам.

Основные сущности и роли:

- Person: Общая сущность для всех участников факультета.
- Student: Учащийся, посещающий курсы и получающий оценки.
- Teacher: Преподаватель, ведущий курсы.
- Staff: Сотрудник факультета, выполняющий административные функции.
- Course: Учебный курс с определенным названием, количеством кредитов и участниками.
- Faculty: Организационная единица, управляющая студентами, преподавателями, сотрудниками и курсами.

Интерфейсы и обязанности:

- IPerson: Определяет базовые атрибуты для идентификации и представления любого участника факультета (getFullName, getId).
- ITeachable: Определяет способность преподавать курс (teachCourse). Реализуется преподавателями (Teacher).
- ILearnable: Определяет способность обучаться, включая зачисление на курсы и получение оценок (enrollCourse, getGrades). Реализуется студентами (Student).

Ключевые взаимодействия и сценарии:

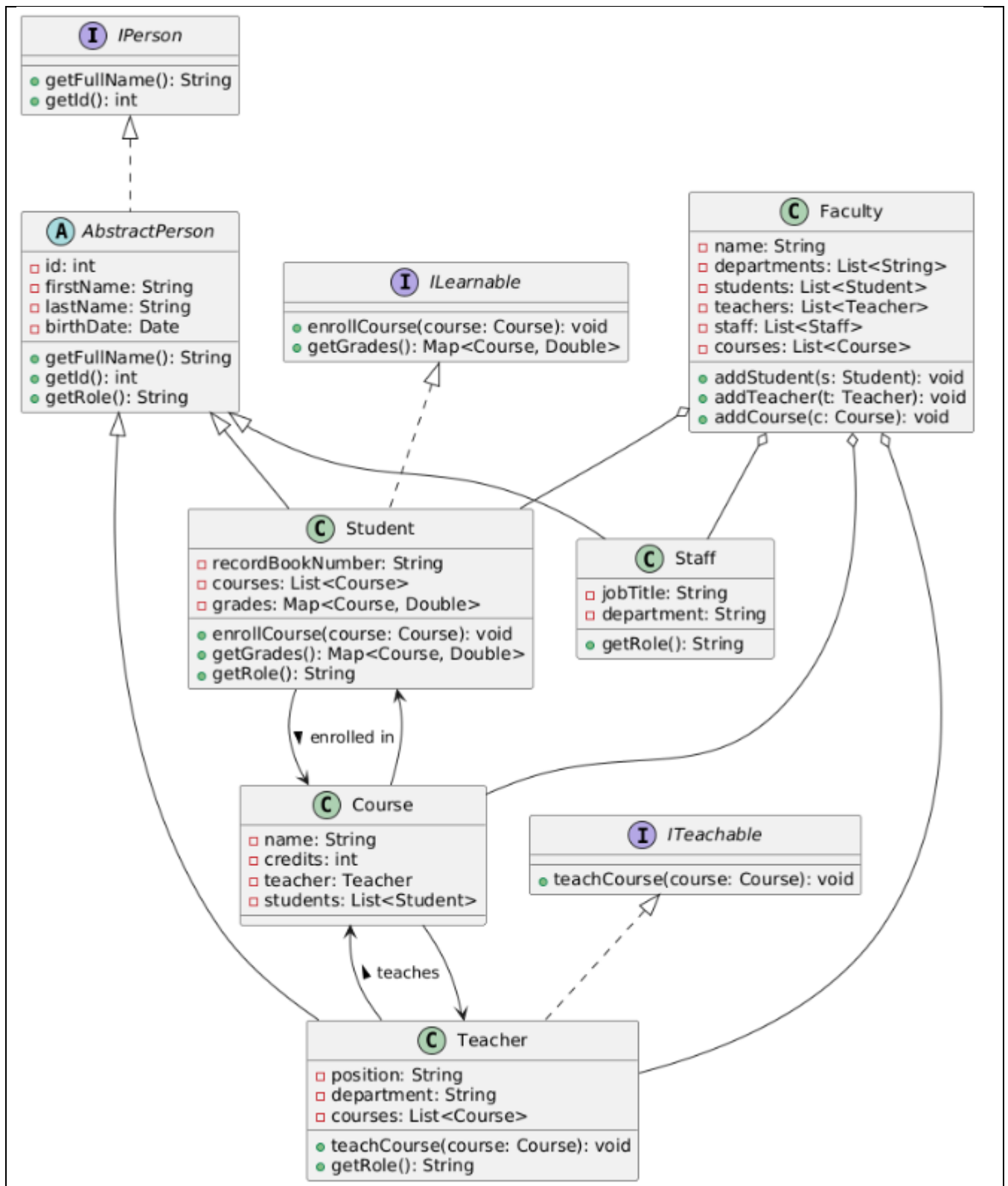
- Зачисление студента на курс: Student.enrollCourse(Course), Course.addStudent(Student).
- Преподавание курса преподавателем: Teacher.teachCourse(Course), Course.setTeacher(Teacher).
- Выставление оценок студенту: (Реализация не указана, но подразумевается) Course.setGrade(Student, grade), Student.getGrades().
- Управление персоналом и курсами: Faculty.add/removeStudent, Faculty.assignTeacherToCourse.

Ограничения и важные требования:

- Каждый человек (Person) должен иметь уникальный идентификатор (id).
- Студент может быть зачислен на несколько курсов.
- Преподаватель может вести несколько курсов.
- Курс может иметь только одного преподавателя.
- Faculty является центральным элементом управления всеми сущностями.

Короткие варианты использования:

- Регистрация нового студента.
- Добавление нового курса.
- Назначение преподавателя на курс.
- Зачисление студента на курс.
- Просмотр списка студентов, записанных на курс.
- Получение информации о преподавателе (список преподаваемых курсов).
- Получение успеваемости студента.



Вариант 8). Продажа товаров супермаркета.

Общая цель системы: Обеспечить эффективное управление и обработку операций, связанных с продажей товаров в супермаркете, включая добавление, удаление товаров из корзины и расчет общей стоимости покупки.

Основные сущности и роли:

- Товар (Product): Представляет собой любой продукт, продаваемый в супермаркете. Разделяется на продукты питания и непродовольственные товары.
- Корзина (Cart): Представляет собой контейнер для выбранных товаров, ожидающих оплаты.
- Супермаркет (неявно): Предоставляет товары и возможность их приобретения.

Интерфейсы и обязанности:

- IProduct: Определяет контракт для всех типов товаров, гарантируя наличие имени, цены и метода отображения информации.
- ICart: Определяет контракт для корзины, гарантируя наличие методов для добавления, удаления товаров и расчета общей стоимости.

Ключевые взаимодействия и сценарии:

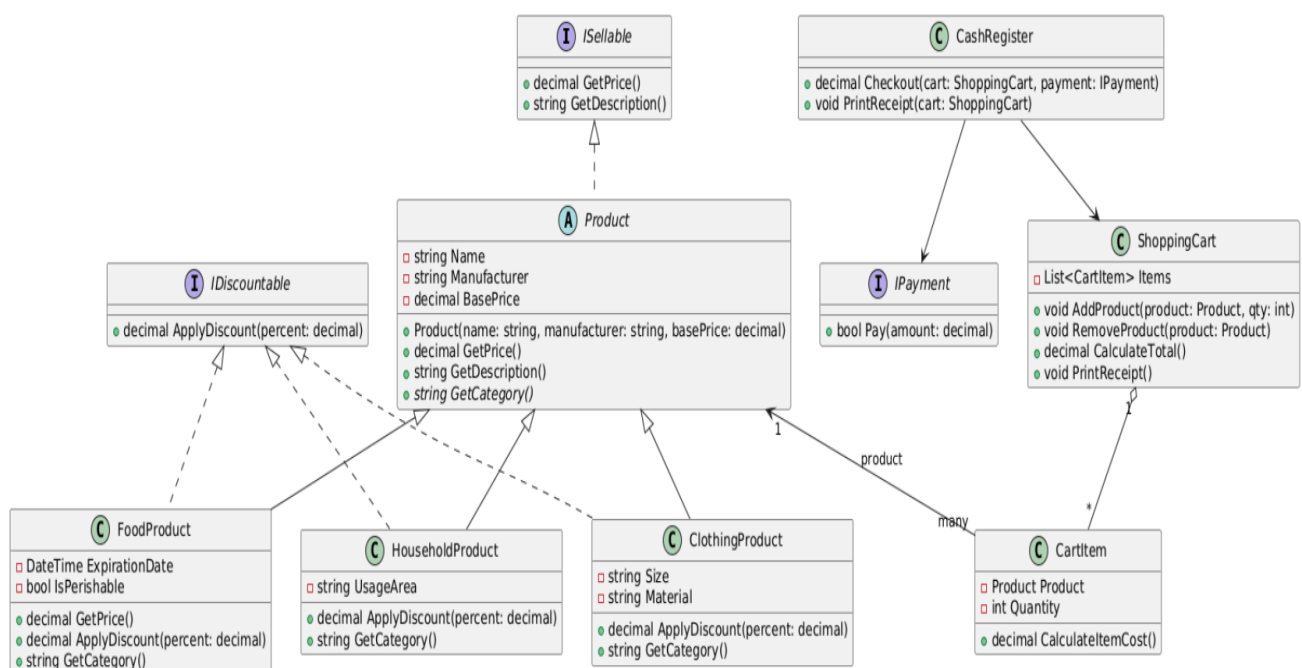
- Добавление товара в корзину: Пользователь выбирает товар, и он добавляется в корзину.
- Удаление товара из корзины: Пользователь удаляет товар из корзины.
- Расчет общей стоимости: Система рассчитывает общую стоимость товаров в корзине.
- Отображение информации о товаре: Система отображает детальную информацию о конкретном товаре (название, цена, срок годности/бренд).

Ограничения и важные требования:

- Различные типы товаров (FoodProduct, NonFoodProduct) должны иметь возможность отображать специфическую для них информацию (например, срок годности для продуктов питания).
- Функциональность корзины должна быть надежной и точной в расчетах.

Короткие варианты использования:

- Добавить продукт: Добавить продукт питания в корзину.
- Удалить продукт: Удалить непродовольственный товар из корзины.
- Рассчитать стоимость: Рассчитать итоговую стоимость товаров в корзине.
- Показать информацию о продукте: Отобразить информацию о выбранном продукте.



Вариант 9). Учет военного состава.

Общая цель системы: Автоматизация учета и управления военнослужащими в рамках военного подразделения.

Основные сущности и роли:

- **IPerson**: Обобщенное представление любой личности, участвующей в системе.
- **MilitaryPerson** (Абстрактный класс): Базовый класс для всех военнослужащих, реализующий основные характеристики личности и возможность прохождения службы.
- **Soldier, Sergeant, Officer**: Конкретные типы военнослужащих, различающиеся по званию и функционалу. **Officer** имеет дополнительный атрибут "должность".
- **MilitaryUnit**: Класс, управляющий списком военнослужащих подразделения.

Интерфейсы и обязанности:

- **IPerson**: Предоставляет стандартизированный доступ к информации о человеке (ФИО, возраст, звание).
- **IServiceable**: Определяет методы для управления службой военнослужащего (начать/закончить).
- **IUnitManagement**: Определяет методы управления списком военнослужащих в подразделении (добавление, удаление, поиск).

Ключевые взаимодействия и сценарии:

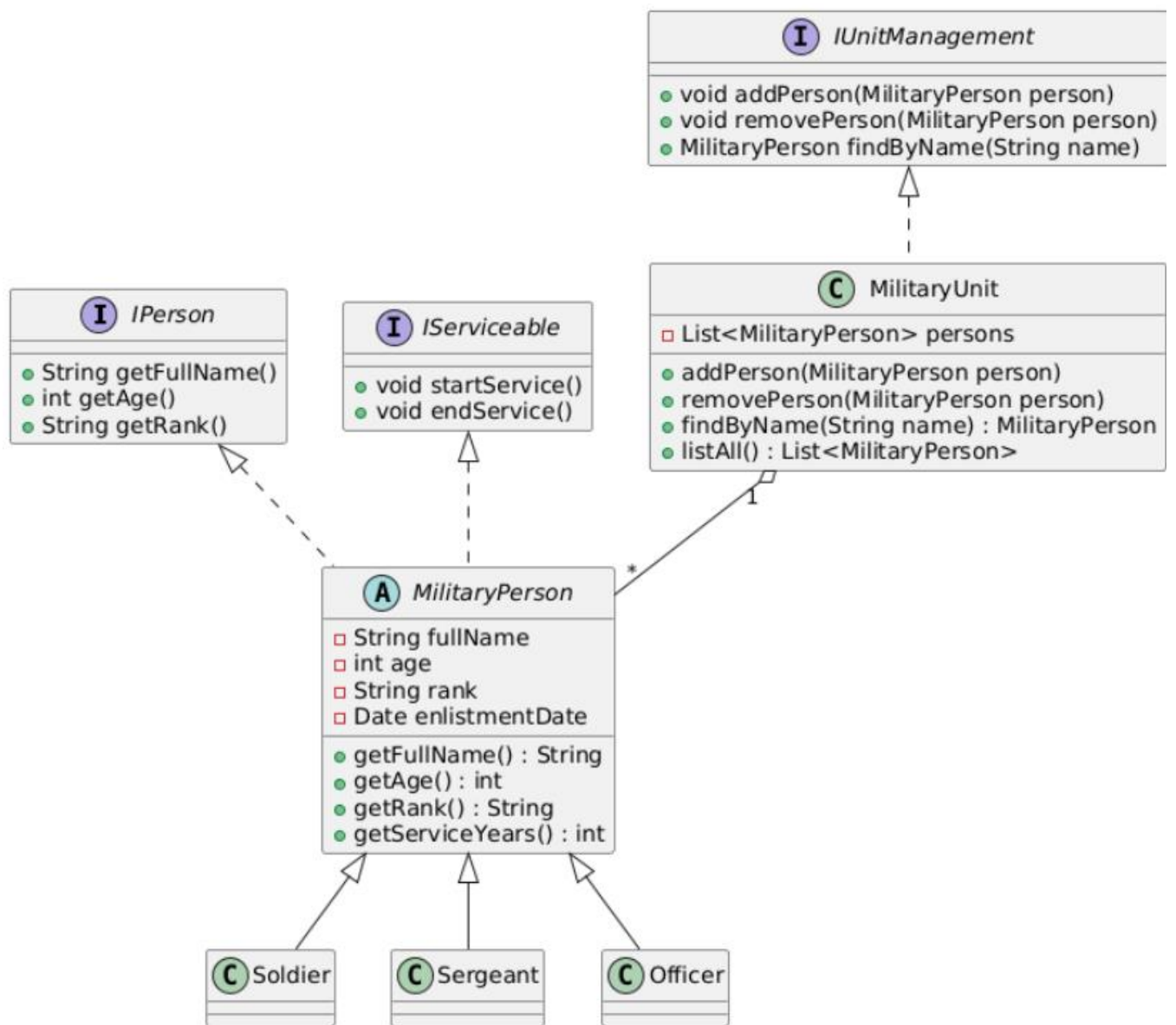
- **Назначение на службу**: Создание экземпляра **Soldier, Sergeant** или **Officer**, добавление его в **MilitaryUnit** с вызовом метода начать службу.
- **Поиск военнослужащего**: Использование методов **MilitaryUnit** для поиска по ФИО или званию.
- **Увольнение со службы**: Удаление военнослужащего из **MilitaryUnit** с вызовом метода закончить службу.
- **Получение информации**: Вывод списка военнослужащих **MilitaryUnit** или информации об отдельном **MilitaryPerson**.

Ограничения и важные требования:

- Необходимо обеспечить корректное хранение и обработку персональных данных военнослужащих.
- Следует предусмотреть возможность расширения системы новыми типами военнослужащих и дополнительными характеристиками.

Короткие варианты использования:

- Добавить нового рядового в подразделение.
- Уволить сержанта Иванова.
- Найти всех офицеров в подразделении.
- Вывести список всех военнослужащих.



Вариант 10). Учет литературы.

Общая цель системы:

Автоматизация процессов учёта и управления информацией о различных видах литературы.

Основные сущности и роли:

- Литература: Общее понятие, включающее книги, журналы и газеты.
- Книга: Отдельный экземпляр книги с уникальным ISBN.
- Журнал: Периодическое издание с номером выпуска.
- Газета: Периодическое издание с указанием издания.
- Пользователь: Лицо, работающее с системой для чтения, сохранения, загрузки и просмотра информации о литературе.

Интерфейсы и обязанности:

- IReadable: Обязывает сущность предоставлять методы для чтения (Read) и получения информации (GetInfo).
- IStorable: Обязывает сущность предоставлять методы для сохранения (Save) и загрузки (Load) данных.

Ключевые взаимодействия и сценарии:

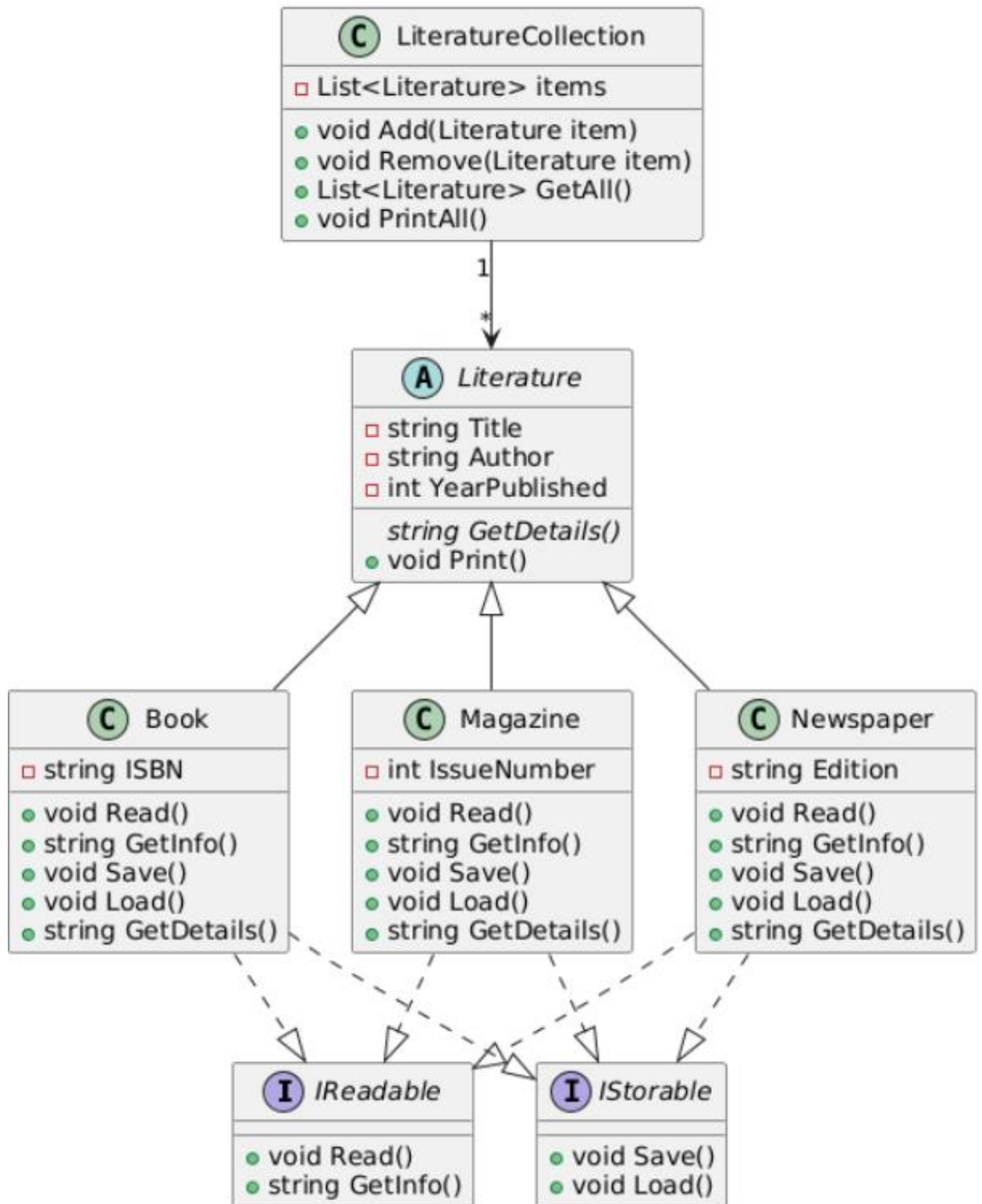
- Пользователь выбирает книгу, журнал или газету.
- Система предоставляет возможность прочитать выбранную литературу.
- Система отображает информацию о выбранной литературе (название, автор, год публикации, и т.д.).
- Пользователь может сохранить информацию о литературе.
- Пользователь может загрузить сохраненную информацию о литературе.

Ограничения и важные требования:

- Необходимо обеспечить хранение и доступ к информации о разных типах литературы.
- Система должна предоставлять возможность получения детальной информации о каждом экземпляре литературы.
- Требуется реализация механизмов сохранения и загрузки данных.

Короткие варианты использования:

- Добавление новой книги в систему.
- Просмотр списка всех доступных журналов.
- Поиск газеты по изданию.
- Сохранение данных обо всех книгах.
- Загрузка ранее сохраненного списка журналов.



Вариант 11). Учет продажи путевок.

Общая цель системы: Автоматизация процесса продажи и учета туристических путевок, а также формирование отчетности.

Основные сущности и роли:

- Путёвка (Voucher): Объект продажи, имеющий различные типы (экскурсионная, санаторная, отдых), характеристики (направление, даты, цена) и уникальный идентификатор.
- Клиент (Client): Покупатель путевки, идентифицируемый по ФИО, паспортным данным и контактам.
- Продажа (Sale): Факт совершения продажи, связывающий клиента, путевку, дату и цену.
- Менеджер путевок (VoucherManager): Ответственен за управление списком доступных путевок, регистрацию продаж и формирование отчетов.

Интерфейсы и обязанности:

- ISellable: Определяет возможность объекта быть проданным (получение цены, осуществление продажи). Обязанность: предоставлять цену и продавать объект клиенту.
- IReportable: Определяет возможность получения информации об объекте для отчета. Обязанность: предоставлять строку с информацией для отчета.
- AbstractVoucher: Базовый класс для всех типов путевок. Обязанности: хранение общих свойств путевки, реализация базовой логики отчетов и продаж, определение абстрактного метода для расчета цены.
- TourVoucher, HealthVoucher, HolidayVoucher: Конкретные типы путевок, наследуемые от AbstractVoucher. Обязанности: Реализация расчета цены в зависимости от специфики каждого типа путевки.

Ключевые взаимодействия и сценарии:

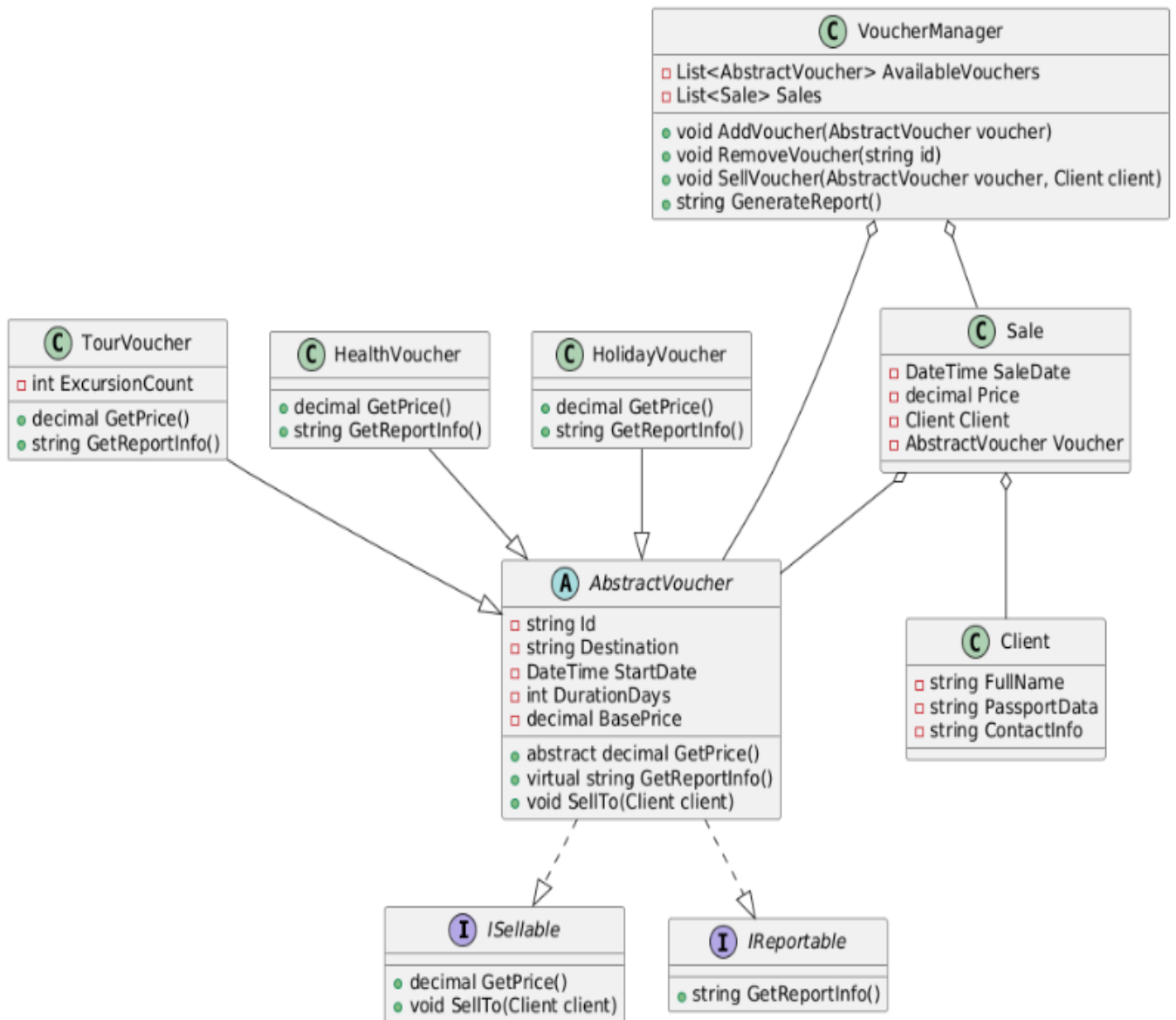
1. Создание путевки: Менеджер добавляет новые путевки различных типов в систему.
2. Отображение доступных путевок: Менеджер/клиент просматривает список доступных путевок.
3. Расчет стоимости путевки: Система рассчитывает стоимость путевки в зависимости от ее типа и заданных параметров.
4. Продажа путевки: Менеджер продает выбранную путевку клиенту.
5. Учет продажи: Система регистрирует информацию о продаже (клиент, путевка, дата, цена).
6. Формирование отчетов: Менеджер формирует отчеты о продажах, доступных путевках и другую информацию.

Ограничения и важные требования:

- Необходимость обеспечения уникальности идентификаторов путевок.
- Корректный расчет стоимости путевок в зависимости от типа и параметров.
- Сохранение истории продаж.
- Возможность формирования отчетов по различным критериям.
- Простота и удобство использования системы для менеджеров.

Короткие варианты использования:

- Добавление новой экскурсионной путевки.
- Просмотр списка санаторных путевок на следующий месяц.
- Продажа путевки на отдых на море клиенту Иванову.
- Формирование отчета о продажах путевок за текущий квартал.
- Получение информации о конкретной путевке для отчета.



Вариант 12). Учет выполненных работ станции техобслуживания.

Общая цель системы:

Обеспечение эффективного учёта, анализа и управления выполненными работами на станции техобслуживания (СТО), включая стоимость, время выполнения, информацию о клиентах, механиках и заказах, для оптимизации работы СТО и повышения её прибыльности.

Основные сущности и роли:

- ServiceStation: Станция техобслуживания, управляющая заказами и отслеживающая доходы.
- Order: Заказ на выполнение работ для конкретного клиента.
- Customer: Клиент, заказывающий работы.
- Work: Абстрактное представление работы, выполняемой на СТО.
- RepairWork, DiagnosticWork, MaintenanceWork: Конкретные типы работ.
- Mechanic: Механик, выполняющий работы.

Интерфейсы и обязанности:

- IBillable: Определяет способность сущности возвращать свою стоимость (GetCost()). Реализуется работами (Work).
- ITimeTrackable: Определяет способность сущности возвращать время выполнения (GetDurationHours()). Реализуется работами (Work).
- IPrintable: Определяет способность сущности предоставлять информацию о себе в виде строки (GetInfo()). Реализуется работами (Work), Order.

Ключевые взаимодействия и сценарии:

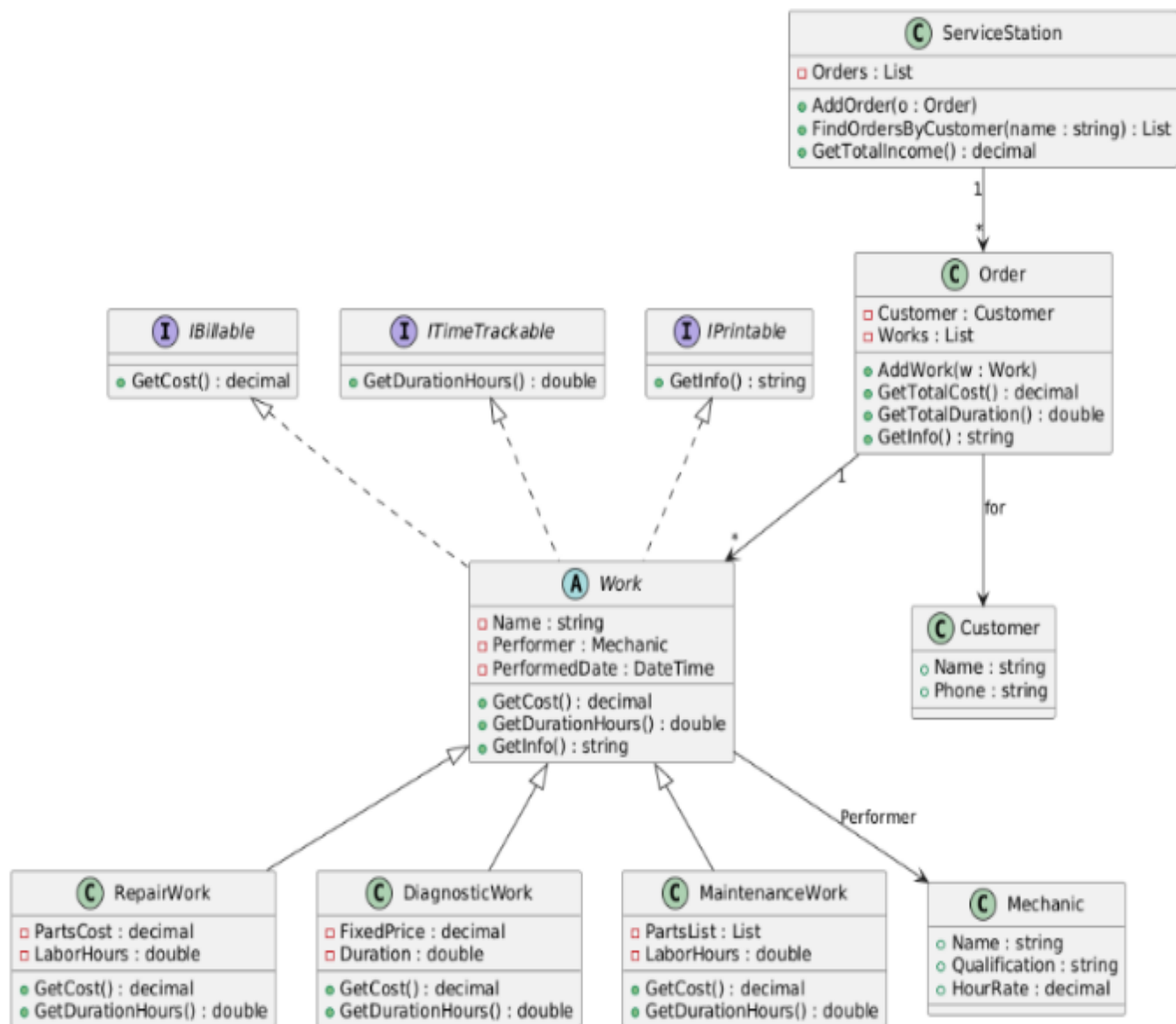
1. Создание заказа: Клиент (Customer) формирует заказ (Order), содержащий список работ (Works).
2. Выполнение работ: Механик (Mechanic) выполняет работы (RepairWork, DiagnosticWork, MaintenanceWork) в рамках заказа.
3. Расчёт стоимости и времени: Система рассчитывает общую стоимость и время выполнения заказа на основе работ и тарифов.
4. Учёт доходов: СТО (ServiceStation) отслеживает доходы от выполненных заказов.
5. Поиск заказов: Система позволяет искать заказы по клиенту (Customer).
6. Получение информации: Система предоставляет информацию о работах, заказах, клиентах и механиках.

Ограничения и важные требования:

- Необходимо обеспечить точный учёт стоимости работ, включая стоимость запчастей и трудозатраты.
- Важно корректно отслеживать время выполнения работ для оценки эффективности механиков.
- Система должна обеспечивать удобный поиск и фильтрацию заказов.
- Необходимо обеспечить возможность получения сводной информации о доходах СТО.

Короткие варианты использования:

- Добавить заказ для клиента.
- Зарегистрировать выполнение ремонтной работы механиком.
- Рассчитать стоимость заказа.
- Получить список заказов для клиента.
- Посмотреть информацию о заказе.
- Рассчитать общий доход СТО за период.



Вариант 13). Медицинское обслуживание пациентов.

Общая цель системы:

Организация и учёт медицинского обслуживания пациентов в больнице, включая управление информацией о пациентах, врачах, медсёстрах, назначение лечения, учёт оказанных услуг и выставление счетов.

Основные сущности и роли:

- Hospital: Больница, управляющая пациентами и персоналом.
- Patient: Пациент, получающий медицинскую помощь.
- Doctor: Врач, назначающий лечение.
- Nurse: Медсестра, оказывающая помощь врачу и пациентам.
- Person: Базовое представление человека в системе (пациент, врач, медсестра).

Интерфейсы и обязанности:

- Identifiable: Предоставляет уникальный идентификатор (GetId()). Реализуется пациентами, врачами и медсестрами.
- ITreatable: Позволяет получать лечение (ReceiveTreatment(string treatment)). Реализуется пациентами.
- IBillable: Позволяет рассчитывать стоимость лечения (CalculateBill()). Реализуется пациентами.

Ключевые взаимодействия и сценарии:

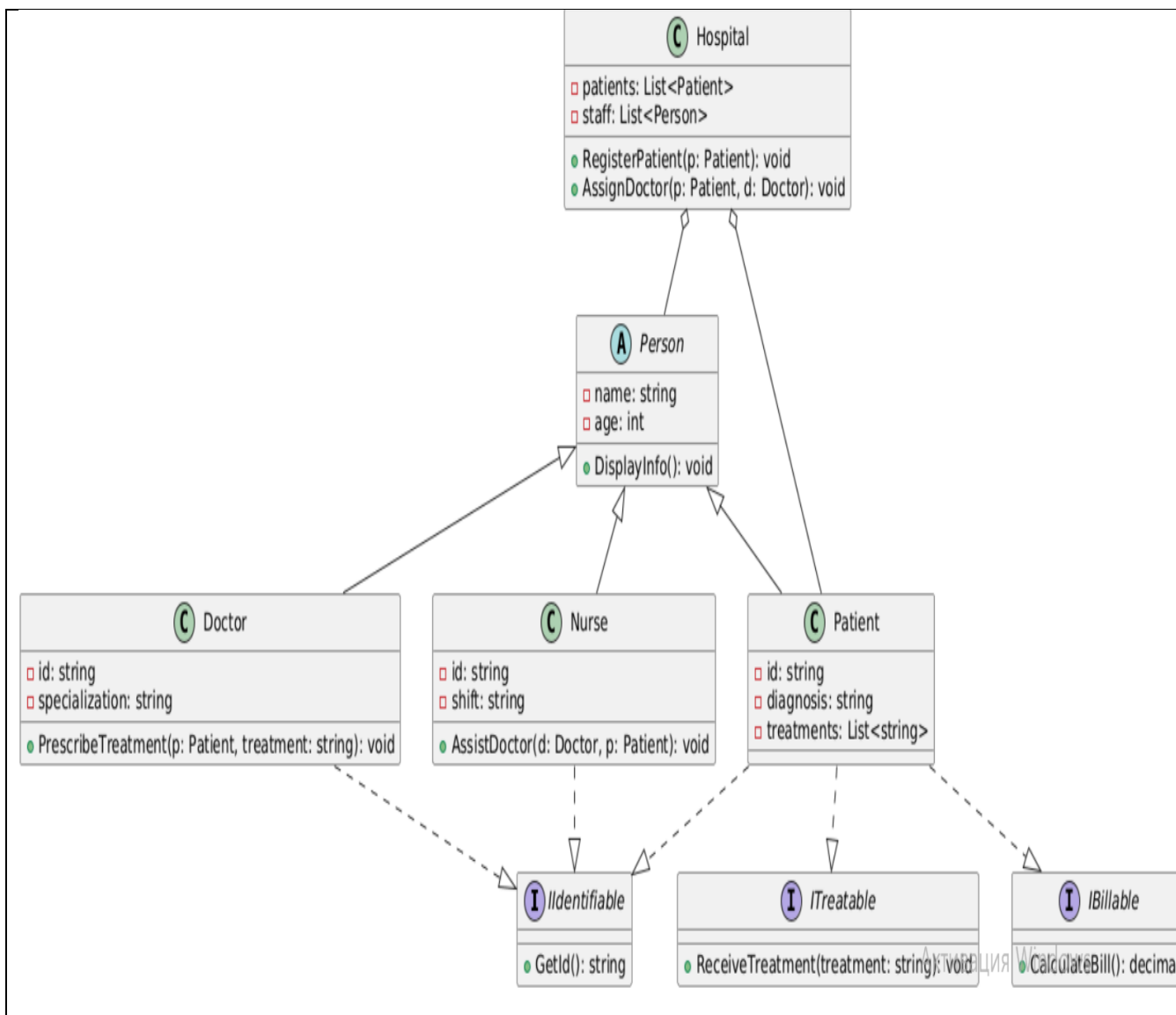
1. Регистрация пациента: Пациент регистрируется в больнице (Hospital).
2. Назначение врача: Пациенту назначается лечащий врач (Doctor).
3. Назначение лечения: Врач (Doctor) назначает лечение (treatment) пациенту (Patient) используя PrescribeTreatment.
4. Оказание помощи: Медсестра (Nurse) помогает врачу (Doctor) в оказании помощи пациенту (Patient).
5. Получение лечения: Пациент (Patient) получает назначенное лечение (treatment) используя ReceiveTreatment.
6. Расчёт стоимости: Система рассчитывает стоимость лечения пациента (Patient) на основе оказанных услуг.

Ограничения и важные требования:

- Необходимо обеспечить уникальную идентификацию пациентов и персонала.
- Важно корректно регистрировать и хранить информацию о назначенном и полученном лечении.
- Система должна обеспечивать возможность расчёта стоимости лечения на основе оказанных услуг.
- Система должна позволять отслеживать взаимосвязи между пациентами, врачами и медсестрами.

Короткие варианты использования:

- Зарегистрировать нового пациента в больнице.
- Назначить врача пациенту.
- Назначить лечение пациенту.
- Зарегистрировать оказание лечения пациенту.
- Рассчитать стоимость лечения для пациента.
- Получить информацию о пациенте, включая его историю лечения.
- Посмотреть список всех пациентов врача.



Вариант 14). Библиотека.

Общая цель системы: Управление библиотечным фондом, предоставление пользователям возможности поиска и получения информации о доступных ресурсах.

Основные сущности и роли:

- Library: Управляет каталогом ресурсов и пользовательскими аккаунтами.
- LibraryItem: Абстрактное представление любого ресурса в библиотеке (книга, журнал, электронная книга).
- Book: Книга с информацией об авторе и количестве страниц.
- Journal: Журнал с указанием номера выпуска.
- EBook: Электронная книга с указанием формата файла.
- User: Зарегистрированный пользователь библиотеки, который может брать и возвращать предметы.

Интерфейсы и обязанности:

- IBorrowable: Описывает предметы, которые можно выдавать пользователям.
 - borrow(User user): Выдает предмет пользователю.
 - returnItem(): Возвращает предмет в библиотеку.
 - isAvailable(): Проверяет, доступен ли предмет для выдачи.
- ISearchable: Описывает предметы, которые можно найти по поисковому запросу.
 - matches(String query): Проверяет, соответствует ли предмет поисковому запросу.

Ключевые взаимодействия и сценарии:

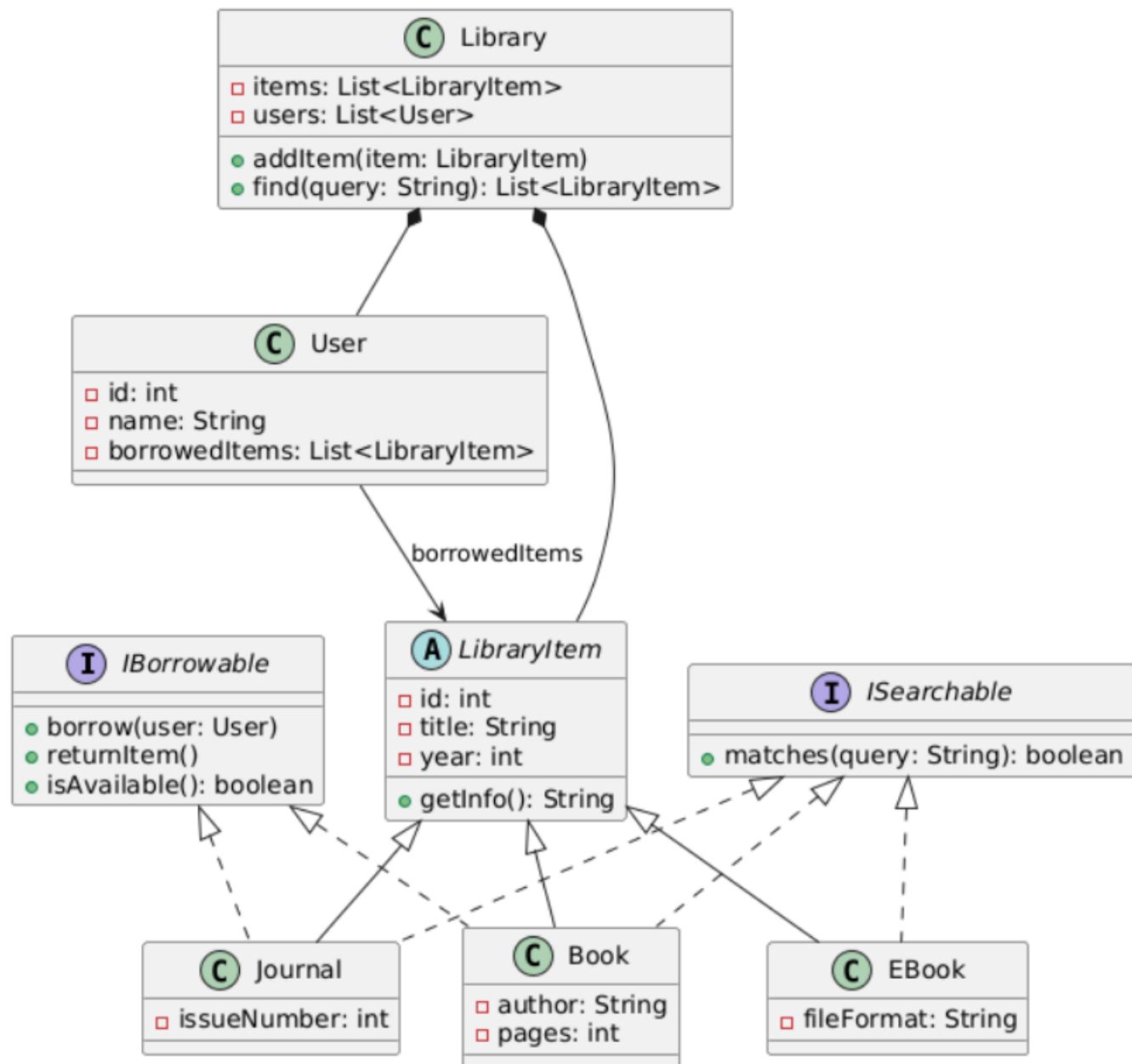
- Пользователь ищет книгу/журнал/электронную книгу по ключевому слову (названию, автору и т.д.).
- Пользователь берет книгу или журнал (если предмет доступен и реализует IBorrowable).
- Пользователь возвращает взятую книгу или журнал.
- Библиотекарь добавляет новые предметы в библиотечный фонд.

Ограничения и важные требования:

- Электронные книги не выдаются пользователям, а предоставляются для онлайн-доступа. (EBook не реализует IBorrowable)
- Реализовать поиск по основным атрибутам сущностей.
- Необходимо поддерживать списки взятых предметов для каждого пользователя.
- Необходимо обеспечивать доступность информации о предметах библиотеки.

Короткие варианты использования:

- Поиск книги по названию.
- Проверка доступности книги для выдачи.
- Выдача книги пользователю.
- Возврат книги в библиотеку.
- Добавление нового журнала в библиотечный фонд.



Вариант 15). Продажа автомобилей

Общая цель системы: Автоматизация процессов продажи автомобилей, включая учет характеристик транспортных средств, применение скидок и оформление сделок купли-продажи.

Основные сущности и роли:

- Vehicle (Автомобиль): Базовый класс, представляющий любой автомобиль, обладает характеристиками (марка, модель, год выпуска, цена) и типом (легковой, грузовой, мотоцикл).
- Car (Легковой автомобиль): Тип автомобиля с дополнительными свойствами (количество дверей) и возможностью применения скидок.
- Truck (Грузовой автомобиль): Тип автомобиля с дополнительными свойствами (грузоподъемность) и возможностью применения скидок.
- Motorcycle (Мотоцикл): Тип автомобиля с дополнительными свойствами (наличие коляски), продается только по базовой цене (без скидок).
- Customer (Покупатель): Физическое или юридическое лицо, приобретающее автомобиль. Обладает информацией об имени и контактном телефоне.
- Sale (Продажа): Транзакция, связывающая покупателя и проданный автомобиль, а также фиксирующая финальную стоимость сделки.

Интерфейсы и обязанности:

- ISellable: Определяет поведение любого продаваемого объекта (получение цены и описания). Обязанности: предоставлять информацию о цене и описании.
- IDisable: Определяет поведение объектов, к которым можно применять скидки. Обязанности: реализовывать логику применения скидок.

Ключевые взаимодействия и сценарии:

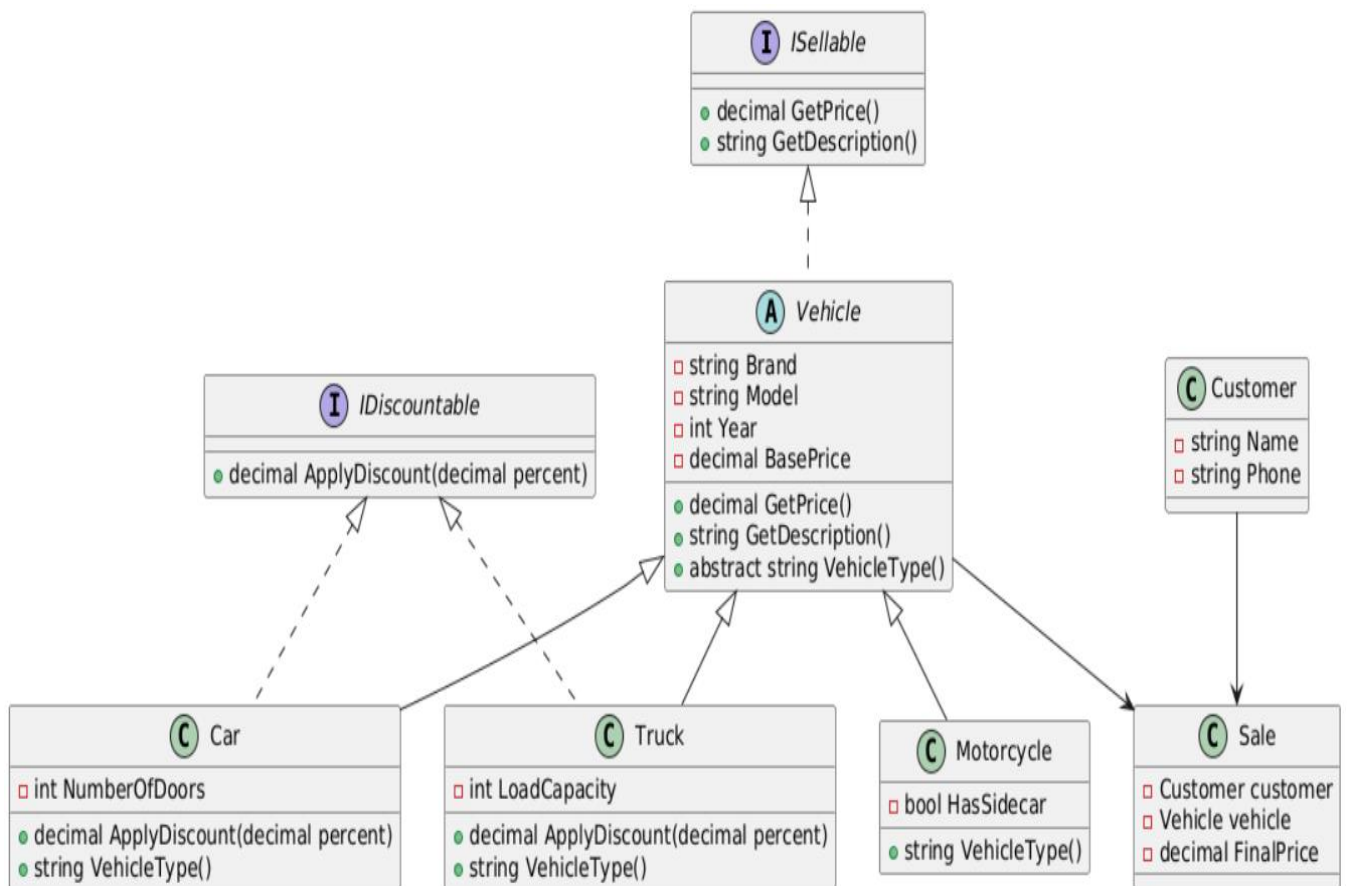
1. Выбор автомобиля: Покупатель выбирает автомобиль (Car, Truck, Motorcycle) из доступного ассортимента.
2. Получение информации об автомобиле: Система предоставляет информацию об автомобиле (описание, цена, характеристики).
3. Применение скидки (для Car и Truck): Система применяет скидку к цене автомобиля (если применимо).
4. Оформление продажи: Система создает запись о продаже, связывая покупателя и автомобиль, а также фиксируя финальную цену.

Ограничения и важные требования:

- Скидки применяются только к легковым и грузовым автомобилям (Car и Truck). Мотоциклы (Motorcycle) продаются только по базовой цене.
- Необходимо обеспечить возможность получения полной информации об автомобиле (описание, цена, характеристики).
- Система должна надежно хранить информацию о продажах и покупателях.

Короткие варианты использования:

- Просмотр каталога автомобилей: Пользователь просматривает доступные для продажи автомобили с указанием их характеристик и цен.
- Расчет стоимости автомобиля со скидкой: Система рассчитывает и отображает финальную стоимость автомобиля с учетом примененной скидки.
- Оформление продажи автомобиля: Система регистрирует сделку купли-продажи автомобиля, включая информацию о покупателе, автомобиле и итоговой сумме.



Вариант 16). Учет продаж через Интернет-магазин.

Общая цель системы:

Автоматизация процессов продажи товаров через интернет, включая управление каталогом, оформление заказов, обработку платежей и предоставление информации о продажах.

Основные сущности и роли:

- Product: Товар, доступный для продажи, с именем, ценой и категорией.
- OrderItem: Позиция в заказе, содержащая информацию о товаре и количестве.
- Order: Заказ, оформленный покупателем, содержащий список позиций, информацию о покупателе и способе оплаты.
- Customer: Покупатель, зарегистрированный в системе, с историей заказов.
- Admin: Администратор системы, имеющий права на управление товарами.
- User: Абстрактное представление пользователя (Customer или Admin).
- Payment Strategies (CreditCardPayment, PayPalPayment, CashOnDeliveryPayment): Различные способы оплаты заказов.

Интерфейсы и обязанности:

- ISellable: Описывает объекты, которые можно продавать (Product). Обязывает реализовывать методы getPrice() и getName().
- IDisableable: Описывает объекты, к которым можно применить скидку (Product). Обязывает реализовывать метод applyDiscount().
- IPayment: Описывает стратегию оплаты. Обязывает реализовывать метод pay().

Ключевые взаимодействия и сценарии:

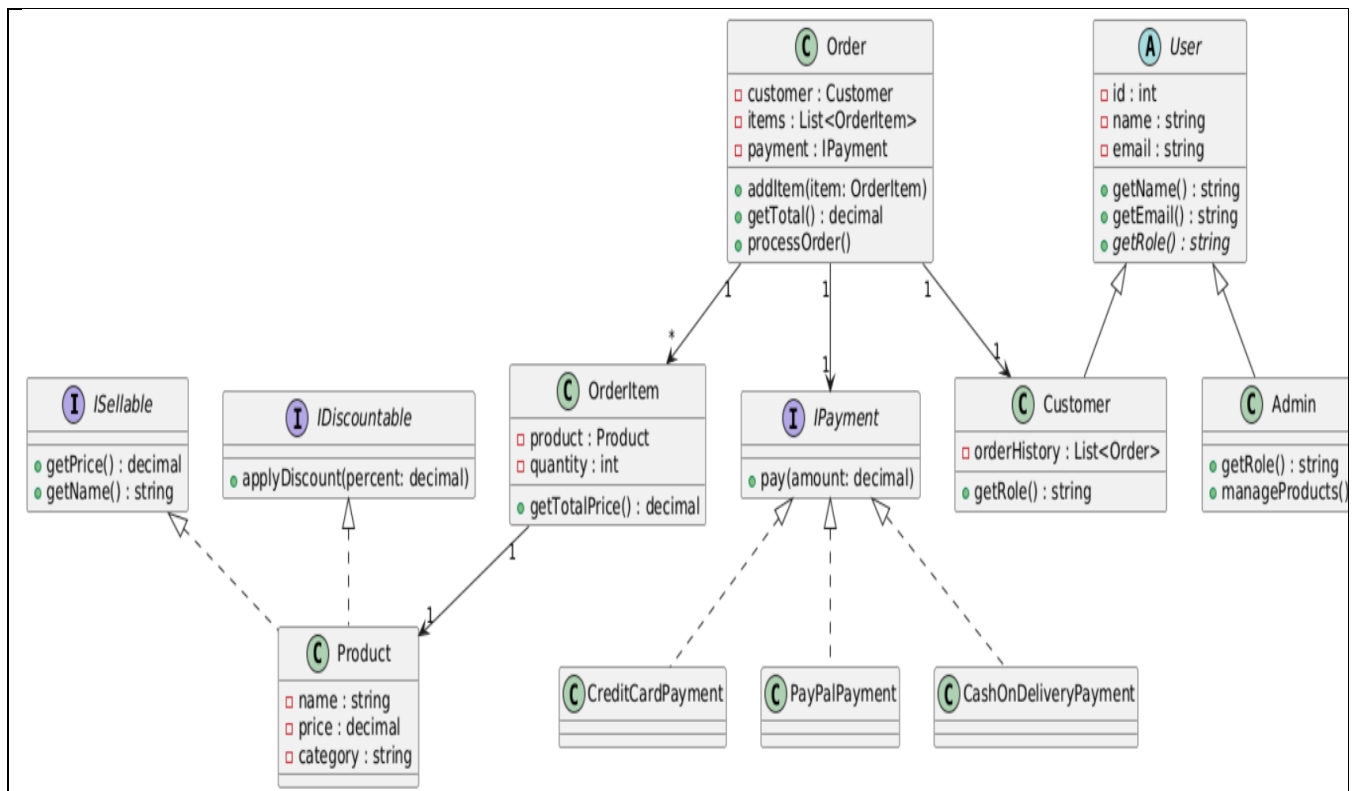
1. Просмотр каталога: Customer просматривает список Product.
2. Добавление в корзину: Customer добавляет Product (в виде OrderItem) в Order.
3. Оформление заказа: Customer выбирает способ оплаты (IPayment) и завершает оформление Order.
4. Обработка платежа: Выбранная стратегия IPayment обрабатывает оплату заказа.
5. Управление товарами: Admin добавляет, редактирует и удаляет Product.
6. Применение скидок: Применение скидок (через applyDiscount) к отдельным товарам Product.

Ограничения и важные требования:

- Система должна обеспечивать безопасную обработку платежей.
- Система должна предоставлять возможность управления каталогом товаров.
- Система должна поддерживать различные способы оплаты.

Короткие варианты использования:

- Покупка товара: Customer добавляет товар в корзину, оформляет заказ и оплачивает его.
- Управление товарами: Admin добавляет новый товар, изменяет цену существующего товара.
- Получение информации о заказе: Customer просматривает историю своих заказов.
- Применение скидки: Admin применяет скидку к выбранной группе товаров.



Вариант 17). Больница

Общая цель системы: Обеспечение эффективной и организованной работы больницы по лечению пациентов и управлению ресурсами.

Основные сущности и роли:

- Пациент: Человек, нуждающийся в лечении.
- Доктор: Медицинский работник, специализирующийся на лечении определенных заболеваний.
- Медсестра: Медицинский работник, обеспечивающий уход за пациентами.
- Административный персонал: Сотрудники, обеспечивающие функционирование больницы.
- Больница: Организация, предоставляющая медицинские услуги.

Интерфейсы и обязанности:

- ITreatable: Объекты (например, пациенты), которые могут получать лечение (receiveTreatment()).
- IWorker: Сотрудники больницы, выполняющие свои рабочие обязанности (workShift(), getPosition()).
- IPayable: Сотрудники, получающие зарплату (calculateSalary()).
- Person: Предоставляет общую информацию о человеке (name, age, getInfo()).

Ключевые взаимодействия и сценарии:

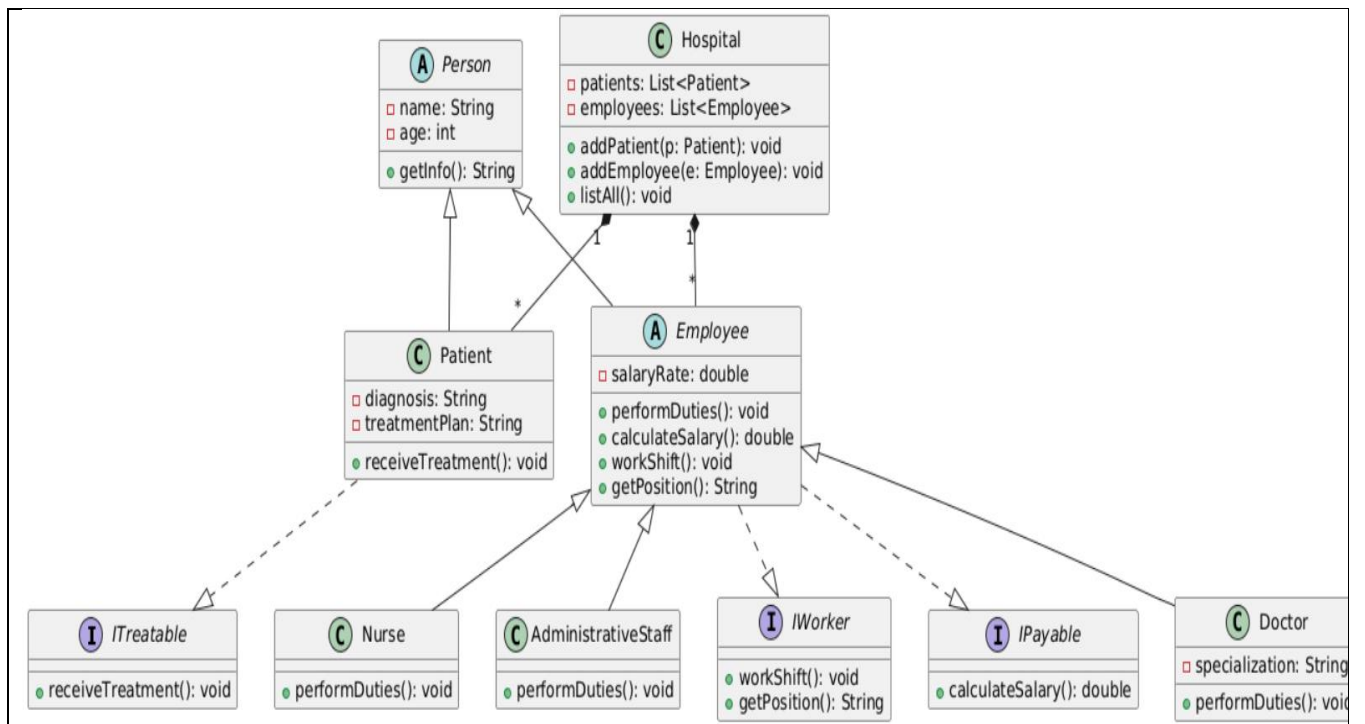
- Прием пациента в больницу (addPatient()).
- Назначение лечения пациенту врачом.
- Уход медсестры за пациентом.
- Выполнение административных задач административным персоналом.
- Начисление заработной платы сотрудникам.
- Отображение списка всех сотрудников и пациентов (listAll()).

Ограничения и важные требования:

- Различные роли сотрудников (врач, медсестра, администратор) имеют разные обязанности.
- Лечение доступно только для пациентов.
- Все сотрудники получают зарплату.
- Персонал работает посменно.

Короткие варианты использования:

- Регистрация нового пациента.
- Назначение врача для пациента.
- Лечение пациента.
- Выплата зарплаты медсестре.
- Поиск информации о сотруднике.
- Отчет о количестве пациентов в больнице.



Вариант 18). Учет проведения военно-морских учений

Общая цель системы: Организация, проведение и анализ результатов военно-морских учений.

Основные сущности и роли:

- **MilitaryUnit** (Военное подразделение): Абстрактное представление любой военной единицы, участвующей в учениях.
- **Ship** (Корабль): Конкретный тип **MilitaryUnit**, обладает возможностями перемещения, боевых действий и отчетности.
- **Submarine** (Подводная лодка): Конкретный тип **Ship**, специализирующийся на подводных операциях.
- **AircraftCarrier** (Авианосец): Конкретный тип **Ship**, предназначенный для базирования и запуска авиации.
- **Exercise** (Учение): Представляет собой спланированное военно-морское учение с участниками, датой и названием.
- **Report** (Отчёт): Документ, содержащий итоговую информацию об учениях, включая сводку, потери и достижения.

Интерфейсы и обязанности:

- **IMovable**: Обеспечивает возможность перемещения объекта в заданные координаты (**MoveTo**).
- **ICombatReady**: Определяет возможности объекта участвовать в боевых действиях (**Attack**, **Defend**).
- **IReportable**: Предоставляет возможность сгенерировать отчёт об объекте (**GenerateReport**).

Ключевые взаимодействия и сценарии:

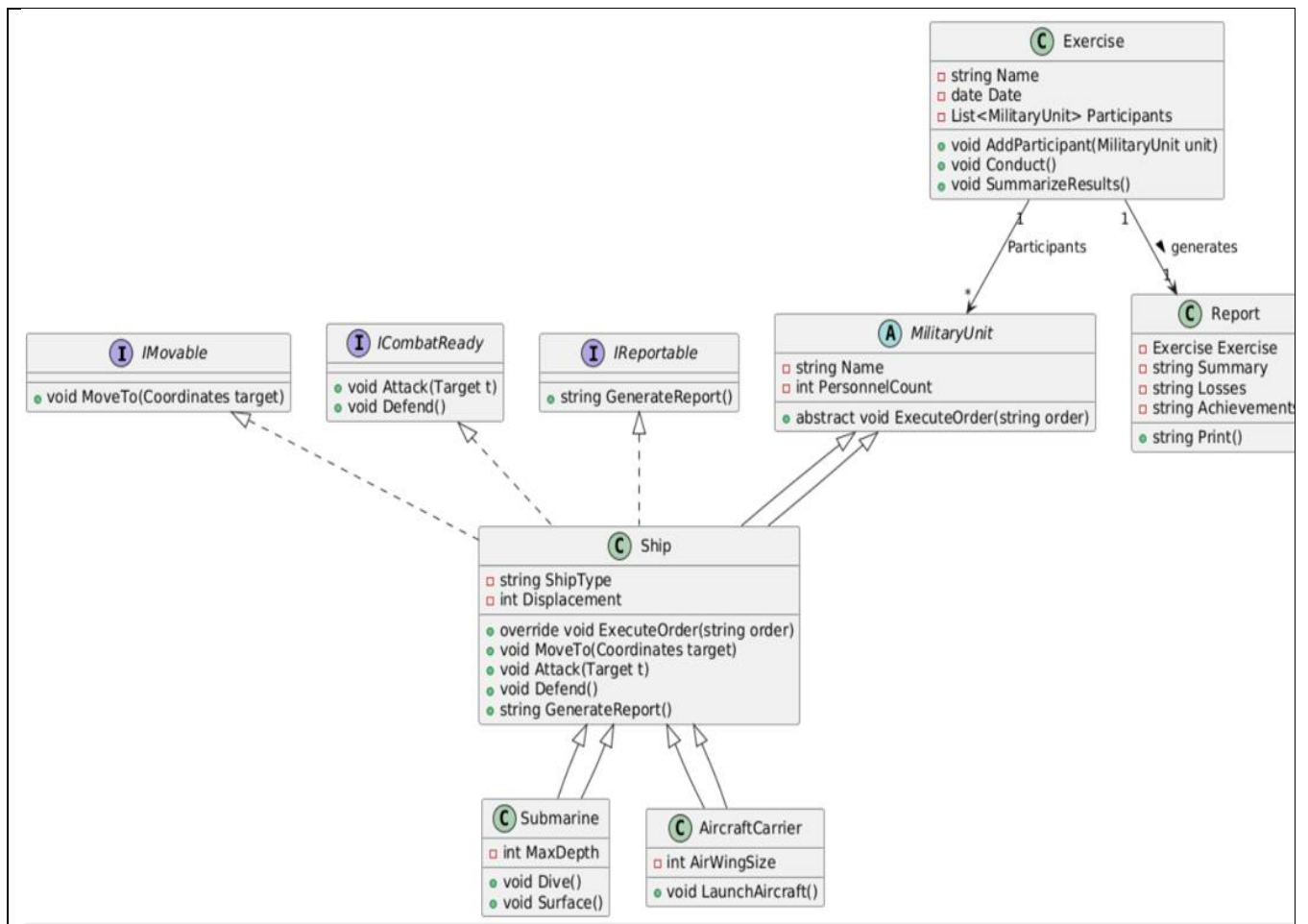
1. Подготовка к учениям: Создание объекта **Exercise**, добавление **MilitaryUnit** (**Ships**, **Submarines**, **AircraftCarriers**) в список участников (**AddParticipant**).
2. Проведение учений: Выполнение учений (**Conduct**), которое включает в себя выдачу приказов (**ExecuteOrder**) различным **MilitaryUnit**, а также использование методов интерфейсов **IMovable** и **ICombatReady**.
3. Подводная лодка: Выполнение команд **Dive**, **Surface** во время учений.
4. Авианосец: Выполнение команд **LaunchAircraft** во время учений.
5. Анализ результатов: Генерация отчёта (**SummarizeResults**), включающего **Report** с информацией о ходе и результатах учений. Печать отчёта (**Print**).

Ограничения и важные требования:

- Система должна обеспечивать учёт различных типов военных подразделений (**Ship**, **Submarine**, **AircraftCarrier**).
- Учения должны моделировать взаимодействие различных подразделений и выполнение боевых задач.
- Система должна предоставлять возможность генерации и анализа отчётов об учениях.

Короткие варианты использования:

- Создание учений: Создать учение "Морской щит 2025" и добавить в него авианосец "Адмирал Кузнецов" и две подводные лодки класса "Варшавянка".
- Проведение учений: Провести учение "Морской щит 2025", отдав приказ кораблям занять заданные позиции и атаковать условного противника.
- Составление отчета: Сформировать и распечатать отчет об учении "Морской щит 2025", включив в него данные о потерях, достижениях и общую сводку.



Вариант 19). Учет оформления туристических туров.

Общая цель системы: Автоматизация процессов управления и учета туристических туров, включая бронирование, ценообразование, транспорт, и отзывы, для повышения эффективности работы туристического агентства.

Основные сущности и роли:

- TourAgency: Туристическое агентство, управляющее списком доступных туров.
- Tour: Абстрактное представление туристического тура (Excursion, BeachTour, Cruise, CityBreak).
- Excursion: Короткая экскурсия в один пункт.
- BeachTour: Пляжный отдых с перелётом.
- Cruise: Круиз по морю.
- CityBreak: Короткое путешествие без отзывов и транспорта.
- Tourist: Клиент, бронирующий туры.

Интерфейсы и обязанности:

- IBookable: Определяет возможность бронирования тура туристом. (Обязанности: Book, CancelBooking)
- IPriced: Определяет наличие цены и возможность применения скидок. (Обязанности: GetPrice, ApplyDiscount)
- ITransportable: Определяет наличие транспорта в туре. (Обязанность: GetTransportType)
- IReviewable: Определяет возможность оставлять отзывы и видеть средний рейтинг тура. (Обязанности: AddReview, GetAverageRating)

Ключевые взаимодействия и сценарии:

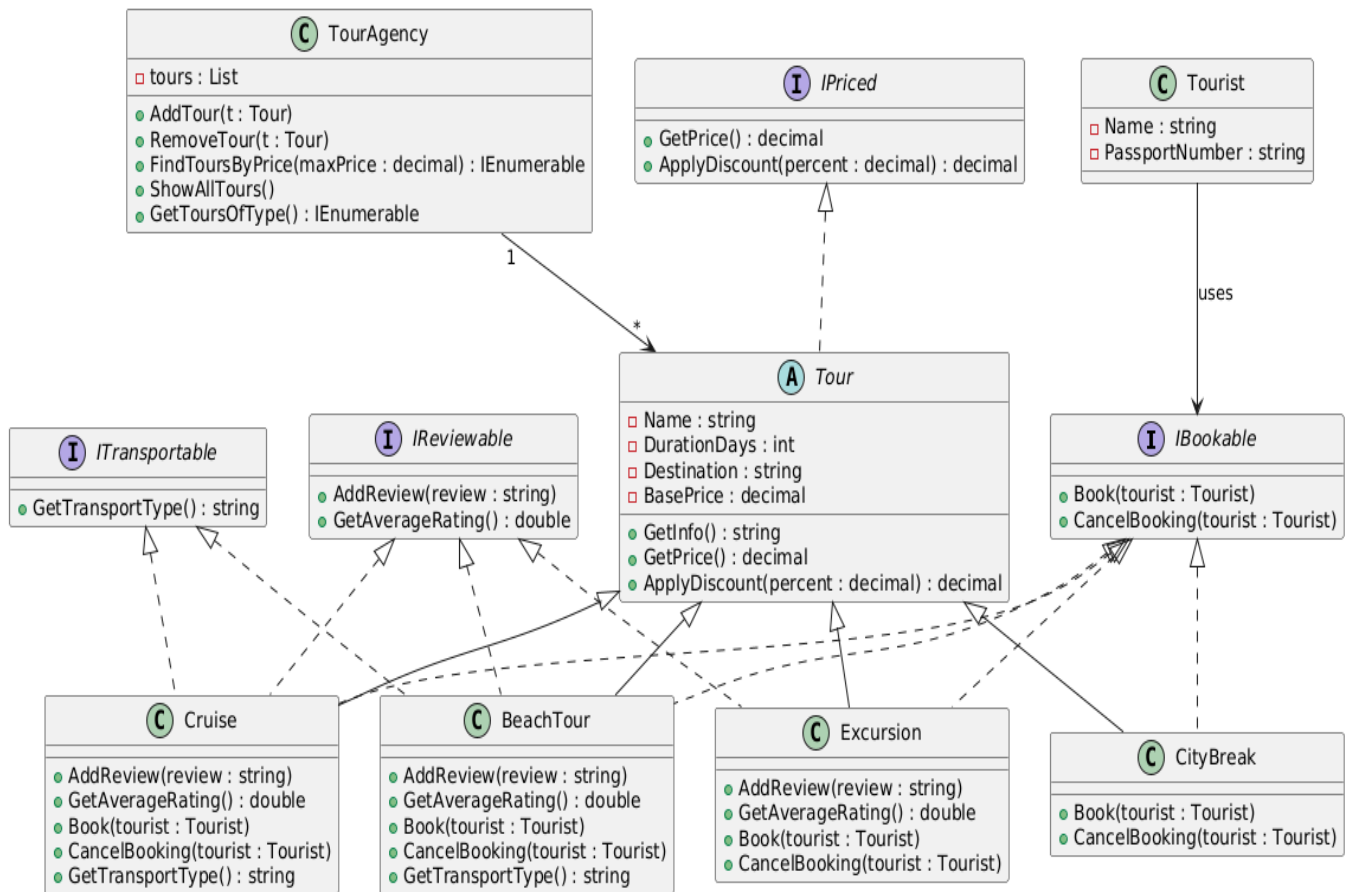
1. Добавление/Удаление туров: TourAgency добавляет и удаляет туры различных типов из своего списка.
2. Поиск туров: TourAgency осуществляет поиск туров по различным критериям (например, максимальной цене, типу тура).
3. Бронирование тура: Tourist бронирует тур, реализующий интерфейс IBookable.
4. Отмена бронирования: Tourist отменяет забронированный тур, реализующий интерфейс IBookable.
5. Оценка и отзывы: Tourist оставляет отзывы о турах, реализующих интерфейс IReviewable.
6. Получение информации о туре: Получение полной информации о туре, включая цену, транспорт (если применимо) и отзывы (если применимо).

Ограничения и важные требования:

- Не все туры поддерживают все функции (например, не все туры включают транспорт или возможность оставить отзыв).
- Цена тура должна быть всегда доступна (реализация IPriced обязательна для всех туров).
- Тип транспорта должен быть четко определен для туров, его включающих.
- Для круизов тип транспорта всегда должен быть "Корабль".

Короткие варианты использования:

- Администратор: Добавляет новый пляжный тур в систему.
- Менеджер: Находит все туры дешевле 50000.
- Клиент: Бронирует экскурсию и оставляет о ней отзыв.
- Клиент: Просматривает доступные круизы.



Вариант 20). Учет представлений цирка.

Общая цель системы: Управление информацией о представлениях цирка, включая планирование, описание, стоимость и продажу билетов.

Основные сущности и роли:

- Цирк (Circus): Организация, владеющая и управляющая представлениями.
- Представление (Performance): Абстрактное понятие, обобщающее различные типы представлений (шоу с животными, акробатические, клоунские).
- Животное представление (AnimalShow): Конкретный тип представления с участием животных.
- Акробатическое представление (AcrobatShow): Конкретный тип представления с акробатами.
- Клоунское представление (ClownShow): Конкретный тип представления с клоунами.
- Билет (Ticket): Документ, дающий право на посещение представления.

Интерфейсы и обязанности:

- IDescribable: Предоставляет текстовое описание сущности (например, название представления, описание билета).
- ICostable: Определяет стоимость сущности (например, стоимость представления, стоимость билета).
- ISchedulable: Определяет дату и время события (например, дату и время представления).

Ключевые взаимодействия и сценарии:

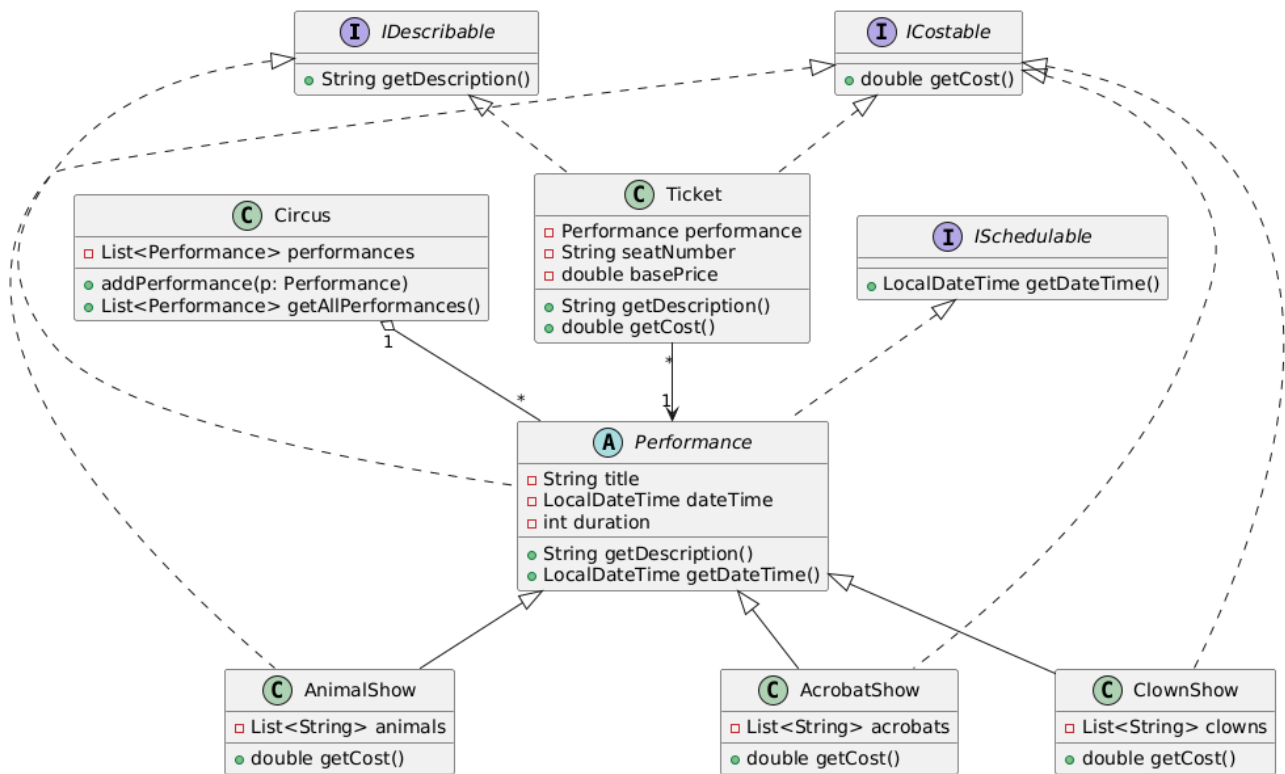
- Добавление представления: Цирк добавляет новое представление (одного из конкретных типов) в свой список.
- Поиск представления: Цирк ищет представление по названию, дате, времени или другим критериям.
- Отображение информации о представлении: Система отображает описание представления, его стоимость и время проведения.
- Продажа билета: Система создает билет на конкретное представление, указывая место и цену. Цена билета может зависеть от представления и места.
- Получение расписания: Система предоставляет расписание всех представлений.

Ограничения и важные требования:

- Разные типы представлений имеют разную логику расчета стоимости.
- Билет должен содержать информацию о представлении, месте и стоимости.
- Система должна обеспечивать удобный поиск и отображение информации о представлениях.

Короткие варианты использования:

- Добавить клоунское представление.
- Показать расписание представлений на следующую неделю.
- Рассчитать стоимость билета на акробатическое представление.
- Получить описание животного представления.
- Найти все представления с клоунами.



Вариант 21). Учет продажи лекарств в аптеке.

Общая цель системы: Автоматизация учета лекарственных средств и продаж в аптеке, повышение эффективности управления запасами и ведения отчетности.

Основные сущности и роли:

- Лекарство (Medicine): Базовая сущность, представляющая лекарственный препарат. Различается на отпускаемые по рецепту (PrescriptionMedicine) и безрецептурные (OTCMedicine).
- Продажа (Sale): Факт продажи определенного количества лекарства в конкретную дату.
- Аптека (Pharmacy): Организация, управляющая запасами лекарств и осуществляющая продажи.
- Фармацевт/Продавец: (Неявно подразумевается, управляет системой через взаимодействие с классом Pharmacy) Осуществляет продажу лекарств, добавляет лекарства в инвентарь.

Интерфейсы и обязанности:

- ISellable: Обеспечивает возможность продажи лекарства, определяет цену и выполняет продажу определенного количества.
- IExpirable: Обеспечивает отслеживание срока годности лекарства.

Ключевые взаимодействия и сценарии:

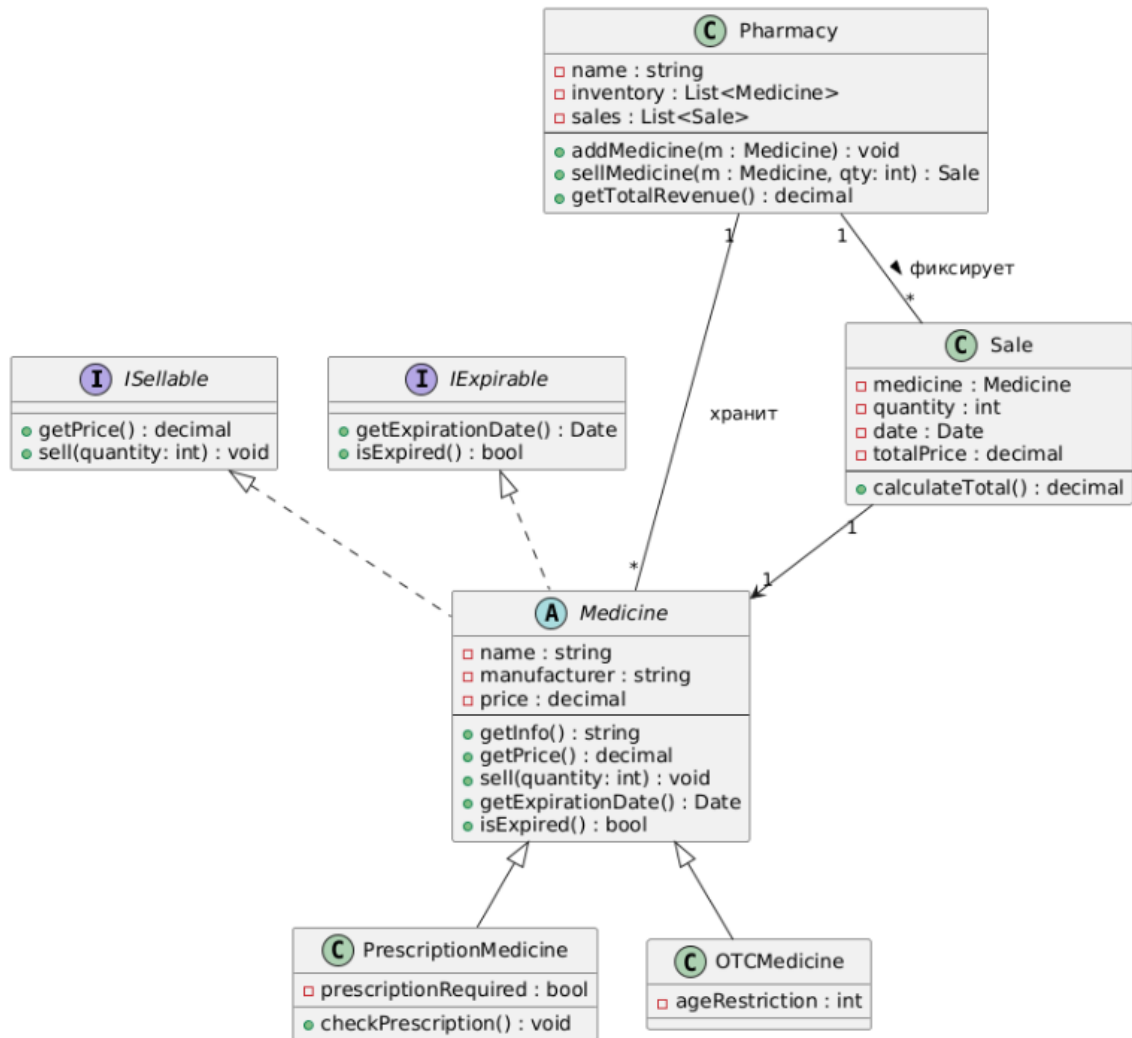
- Добавление лекарства в аптеку (Pharmacy.addMedicine)
- Продажа лекарства (Pharmacy.sellMedicine) - включает проверку рецепта (PrescriptionMedicine.checkPrescription) и ограничений по возрасту (OTCMedicine.ageRestriction).
- Вычисление общей выручки (Pharmacy.getTotalRevenue).
- Проверка срока годности лекарства (IExpirable.isExpired).
- Вывод информации о лекарстве (Medicine.getInfo).

Ограничения и важные требования:

- Учет лекарств, отпускаемых по рецепту, должен требовать подтверждения наличия рецепта (PrescriptionMedicine).
- Необходимо учитывать срок годности лекарств (IExpirable).
- Продажа безрецептурных лекарств может быть ограничена по возрасту (OTCMedicine).
- Валидное логирование продаж (sale), в том числе дата, количество, общая стоимость и конкретное лекарство.

Короткие варианты использования:

- Фармацевт добавляет новую партию лекарства в аптеку.
- Фармацевт продает лекарство по рецепту, проверяя рецепт.
- Фармацевт продает безрецептурное лекарство, проверяя возраст покупателя (если необходимо).
- Система автоматически уведомляет о лекарствах с истекающим сроком годности.
- Менеджер получает отчет об общей выручке аптеки за период.



Вариант 22). Учет абонентской платы.

Общая цель системы: Автоматизация процесса учета и списания абонентской платы для различных видов подписок (интернет, мобильная связь, ТВ) у абонентов.

Основные сущности и роли:

- Абонент (Subscriber): Пользователь, владеющий подписками и платящий за них.
- Подписка (Subscription): Услуга, предоставляемая абоненту на платной основе (интернет, мобильная связь, ТВ).
- Интернет-подписка (InternetSubscription): Конкретный тип подписки, предоставляющий доступ к интернету с определенными параметрами (трафик, скорость).
- Мобильная подписка (MobileSubscription): Конкретный тип подписки, предоставляющий услуги мобильной связи (минуты, SMS, трафик).
- ТВ-подписка (TVSubscription): Конкретный тип подписки, предоставляющий доступ к телевизионным каналам.

Интерфейсы и обязанности:

- IPayable: Определяет возможность объекта принимать оплату (Абонент).
- IPrintable: Определяет возможность объекта выводить информацию о себе (Абонент, Подписка).
- IBillable: Определяет возможность объекта вычислять абонентскую плату (Подписка).

Ключевые взаимодействия и сценарии:

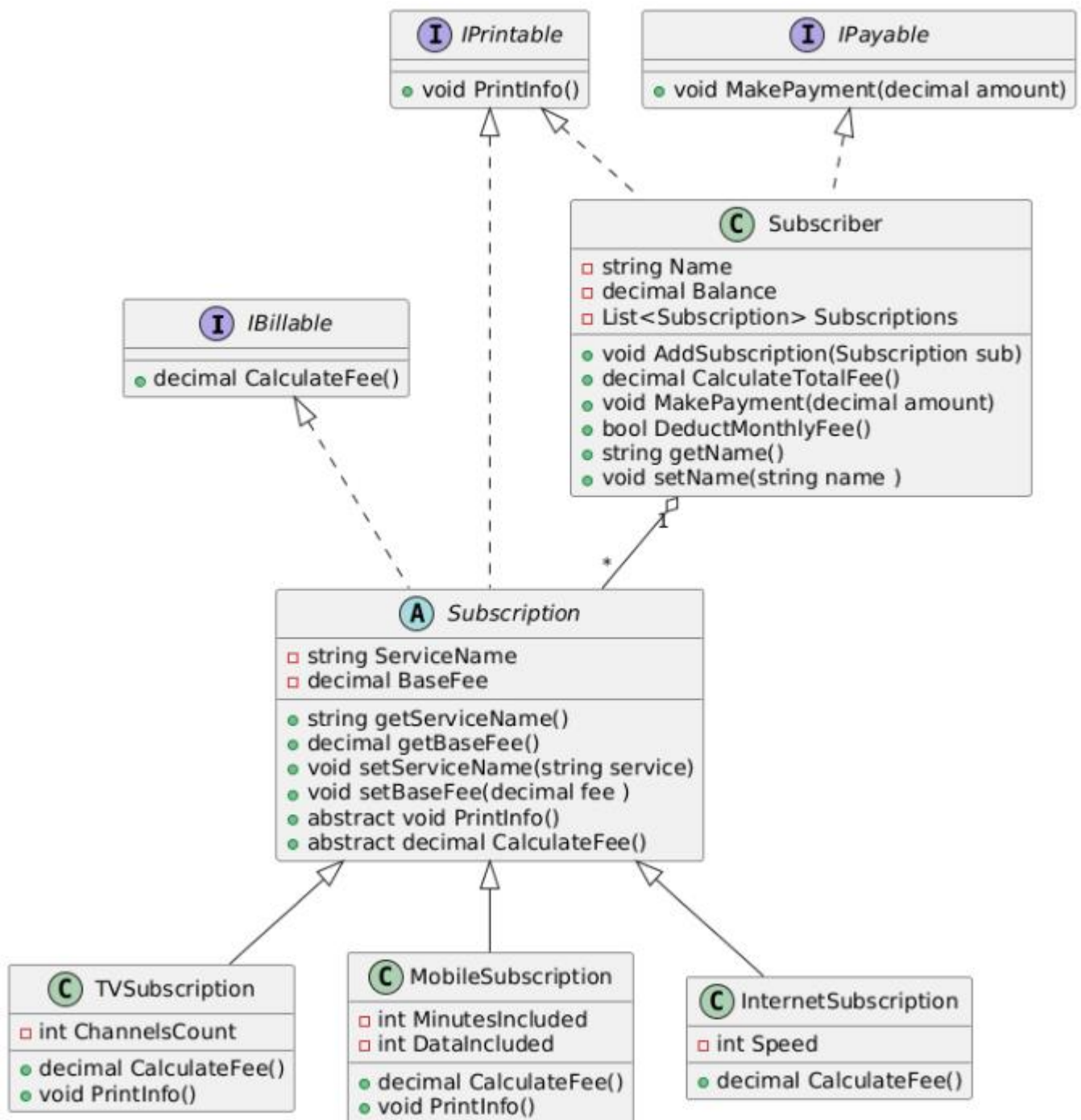
1. Регистрация подписки: Абонент оформляет подписку определенного типа.
2. Расчет абонентской платы: Система вычисляет абонентскую плату для каждой подписки и общую сумму для абонента.
3. Внесение оплаты: Абонент вносит деньги на свой баланс.
4. Списание абонентской платы: Система автоматически списывает абонентскую плату с баланса абонента.
5. Вывод информации: Система предоставляет информацию об абоненте (имя, баланс, подписки) и подписках (тип, параметры, стоимость).

Ограничения и важные требования:

- Необходимо корректно рассчитывать абонентскую плату для разных типов подписок, учитывая их параметры.
- Необходимо обеспечить возможность пополнения баланса абонента.
- Необходимо отслеживать состояние баланса абонента и уведомлять о недостаточном количестве средств для списания абонентской платы.
- Необходимо обеспечить вывод информации об абонентах и их подписках в удобном формате.

Короткие варианты использования:

- Добавить новую подписку абоненту.
- Пополнить баланс абонента.
- Рассчитать общую абонентскую плату для абонента.
- Списывать абонентскую плату с баланса абонента.
- Вывести информацию об абоненте и его подписках.



Вариант 23). Стоматологическая поликлиника

Общая цель системы: Автоматизация и управление процессами оказания стоматологических услуг в поликлинике, обеспечивающая эффективное взаимодействие между сотрудниками, пациентами и предоставляемыми услугами.

Основные сущности и роли:

- Пациент: Клиент поликлиники, имеющий медицинскую карту и нуждающийся в стоматологических услугах.
- Врач: Сотрудник поликлиники, оказывающий стоматологические услуги определенной специализации.
- Медсестра: Сотрудник поликлиники, оказывающий помощь врачу в оказании стоматологических услуг.
- Администратор: Сотрудник поликлиники, отвечающий за регистрацию пациентов, запись на прием и ведение документации.
- Услуга: Конкретная стоматологическая процедура, предоставляемая пациенту (например, осмотр, удаление зуба, чистка зубов).
- Поликлиника: Организация, предоставляющая стоматологические услуги, содержащая штат сотрудников и список доступных услуг.

Интерфейсы и обязанности:

- IPrintable: Обеспечивает возможность получения информации о сущности (Имя, Возраст, GetInfo()). Реализован для всех, кого можно представить (Люди).
- IMedicalService: Определяет метод ProvideService(Patient), описывающий процесс оказания медицинской услуги пациенту. Реализован Врачами и Стоматологическими Услугами.

Ключевые взаимодействия и сценарии:

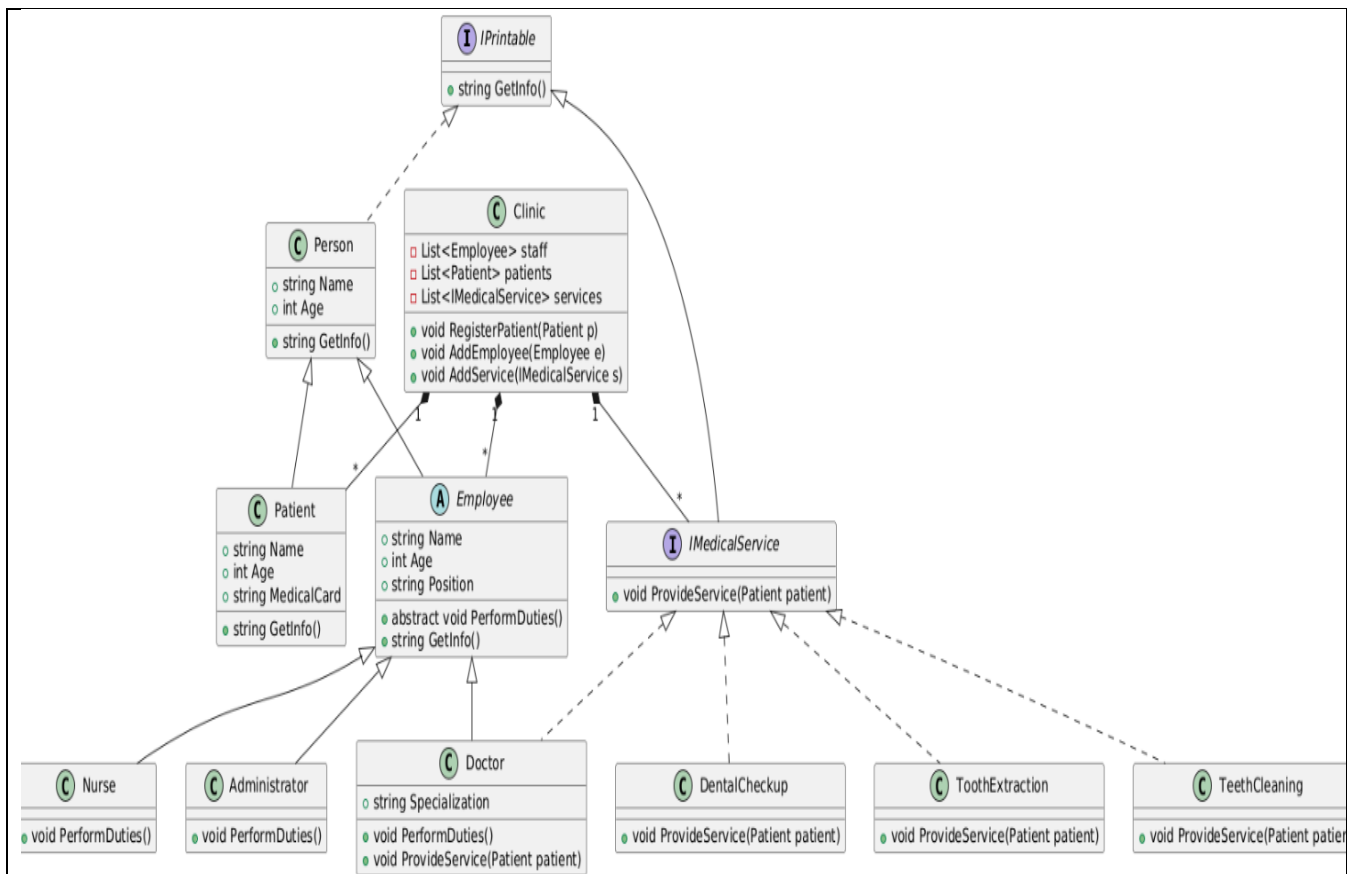
- Запись на прием: Пациент записывается на прием к врачу через администратора.
- Оказание услуги: Врач, с помощью медсестры, оказывает пациенту выбранную услугу.
- Ведение медицинской карты: Врачи и администраторы вносят информацию об оказанных услугах и состоянии пациента в медицинскую карту.
- Управление сотрудниками: Поликлиника управляет штатом сотрудников, включая назначение обязанностей и контроль их выполнения.

Ограничения и важные требования:

- Система должна обеспечивать конфиденциальность медицинской информации.
- Система должна поддерживать различные специализации врачей.
- Система должна позволять ведение истории посещений и оказанных услуг для каждого пациента.

Короткие варианты использования:

- Регистрация нового пациента.
- Запись пациента на прием к врачу.
- Проведение осмотра пациента врачом.
- Оказание стоматологической услуги пациенту (например, удаление зуба).
- Просмотр истории посещений пациента.
- Формирование отчетов о оказанных услугах.



Вариант 24). Каршеринг.

Общая цель системы: Предоставить пользователям платформу для краткосрочной аренды транспортных средств (автомобилей, электромобилей, скутеров), обеспечивая удобный доступ к транспорту без необходимости владения им.

Основные сущности и роли:

- Транспортное средство (Vehicle): Основной объект аренды (автомобиль, электромобиль, скутер).
- Пользователь (User): Лицо, имеющее возможность арендовать транспортные средства.
- Каршеринг Сервис (CarSharingService): Организация, управляющая парком транспортных средств и предоставляющая услуги аренды.

Интерфейсы и обязанности:

- IRentable: Определяет возможность аренды транспортного средства, включает выдачу в аренду (rent) и возврат (returnCar).
- IMaintainable: Определяет необходимость технического обслуживания транспортного средства, включает выполнение обслуживания (performMaintenance).
- Vehicle: Абстрактное представление транспортного средства, содержит общие свойства и определяет абстрактное поведение drive.

Ключевые взаимодействия и сценарии:

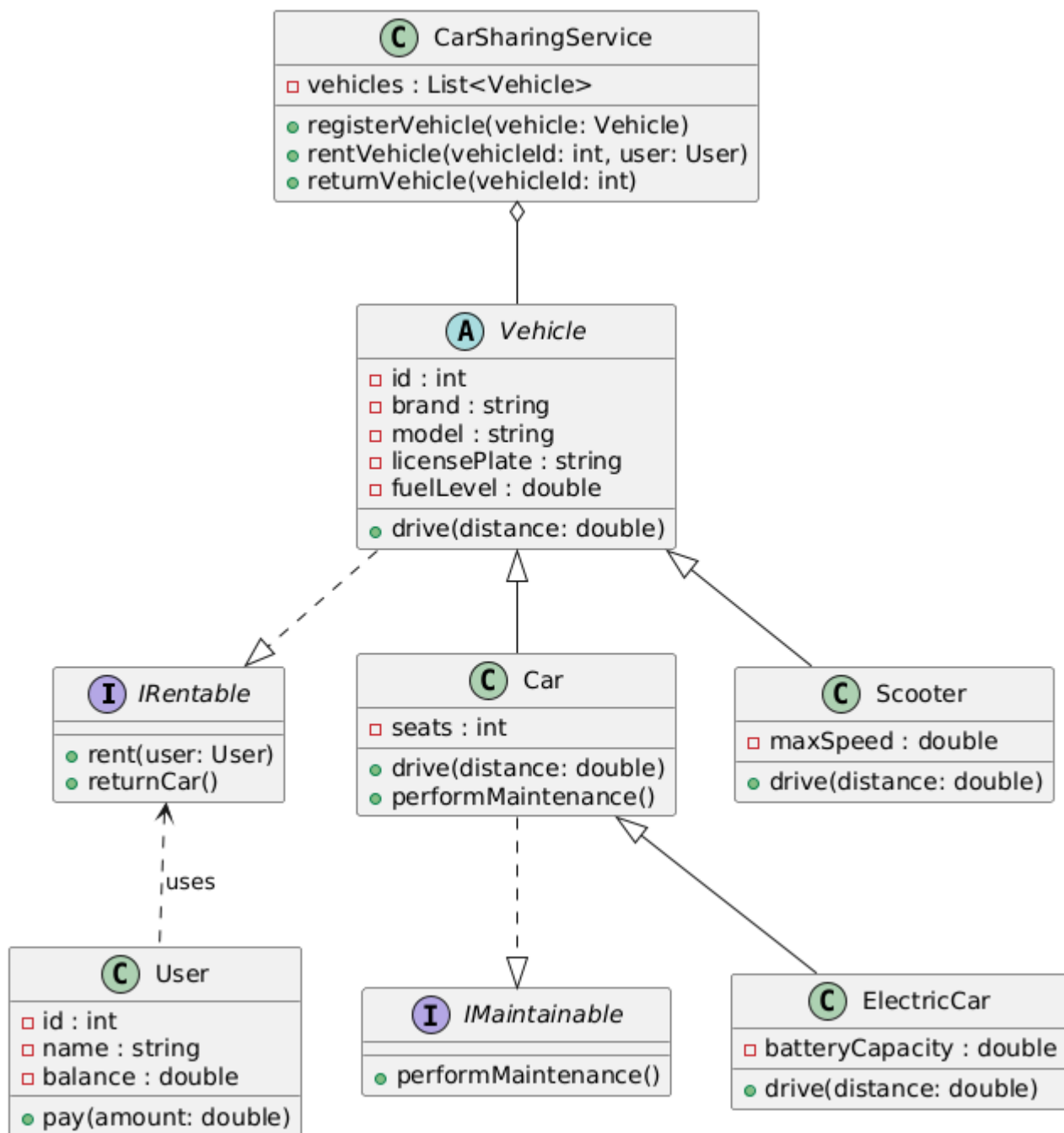
1. Регистрация транспортного средства: CarSharingService регистрирует новые транспортные средства (Car, ElectricCar, Scooter).
2. Аренда транспортного средства: User запрашивает аренду транспортного средства по vehicleId у CarSharingService. CarSharingService проверяет доступность и, при успехе, передает управление транспортным средством User.
3. Использование транспортного средства: User использует транспортное средство, вызывая метод drive. В зависимости от типа транспортного средства, поведение drive может различаться (расход топлива, заряда батареи).
4. Возврат транспортного средства: User возвращает транспортное средство CarSharingService, завершая аренду.
5. Обслуживание транспортного средства: CarSharingService проводит обслуживание транспортных средств, реализуя интерфейс IMaintainable.
6. Оплата: User оплачивает аренду транспорта, списывая средства со своего баланса через метод pay.

Ограничения и важные требования:

- Наличие достаточного баланса у пользователя для аренды.
- Проверка доступности транспортного средства перед арендой.
- Необходимость выполнения обслуживания для поддержания транспортных средств в рабочем состоянии.
- Корректный расчет стоимости аренды и расхода топлива/энергии.

Короткие варианты использования:

- Аренда автомобиля: Пользователь арендует доступный автомобиль для поездки.
- Зарядка электромобиля: После поездки электромобиль ставится на зарядку.
- Техническое обслуживание: Автомобиль проходит плановое техническое обслуживание.
- Пополнение баланса: Пользователь пополняет свой баланс для совершения поездок.



Вариант 25). Автоматизация работы кафедры.

Общая цель системы:

Автоматизировать и оптимизировать основные процессы кафедры, включая управление персоналом, учебным процессом, научно-исследовательской деятельностью и административными задачами.

Основные сущности и роли:

- Кафедра (Department): Центральная сущность, управляющая сотрудниками, курсами и деятельностью в целом.
- Сотрудник (Employee): Общий термин для преподавателей, научных сотрудников и администраторов.
 - Преподаватель (Teacher): Проводит занятия и разрабатывает методические материалы.
 - Научный сотрудник (Researcher): Проводит исследования.
 - Администратор (Administrator): Выполняет административные и управленческие функции.
- Студент (Student): Участник учебного процесса, посещает курсы и сдает экзамены.
- Дисциплина (Course): Учебный курс, включающий студентов и преподавателя.

Интерфейсы и обязанности:

- IPerson: Определяет базовые атрибуты для всех людей (имя, контакты). Реализуется сотрудниками и студентами.
- ITeaching: Определяет методы для проведения занятий и разработки учебных материалов. Реализуется преподавателями.
- IAdministrative: Определяет методы для административных и управленческих задач. Реализуется администраторами.
- Employee (Абстрактный класс): Определяет общие атрибуты и обязанности для всех сотрудников (имя, должность, email, обязанности).

Ключевые взаимодействия и сценарии:

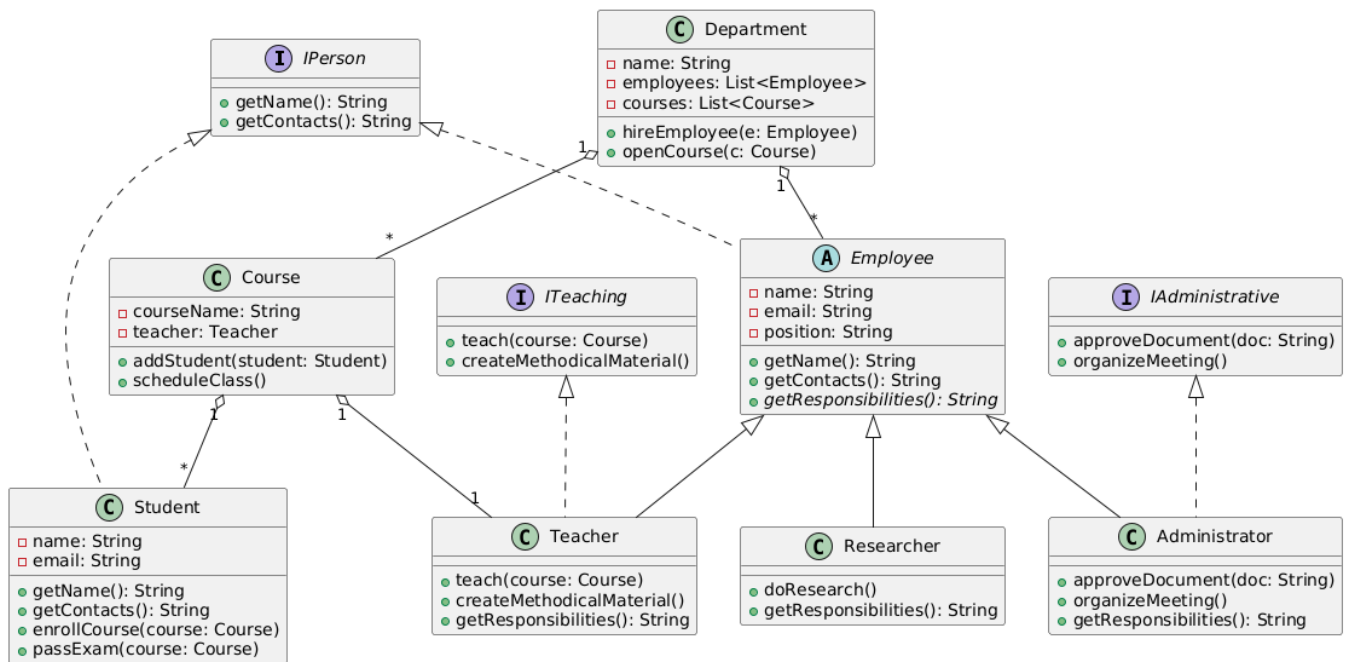
- Наем сотрудника: Кафедра нанимает новых сотрудников (преподавателей, исследователей, администраторов).
- Открытие курса: Кафедра открывает новые учебные курсы и назначает преподавателя.
- Зачисление на курс: Студенты записываются на выбранные курсы.
- Проведение занятий: Преподаватели проводят занятия и разрабатывают методические материалы.
- Научные исследования: Научные сотрудники проводят исследования.
- Административные задачи: Администраторы утверждают документы и организуют собрания.

Ограничения и важные требования:

- Необходимо обеспечить разграничение прав доступа для разных ролей (студент, преподаватель, администратор).
- Система должна обеспечивать хранение и обработку персональных данных в соответствии с требованиями безопасности.
- Необходима интеграция с существующими системами университета (например, система управления обучением).
- Система должна быть масштабируемой для поддержки роста кафедры и количества студентов.

Короткие варианты использования:

- Администратор добавляет нового преподавателя на кафедру.
- Преподаватель создает новый курс и назначает даты занятий.
- Студент записывается на выбранный курс.
- Научный сотрудник публикует результаты исследования.
- Администратор формирует расписание занятий на семестр.



Контрольные вопросы

- 1 Особенности абстрактного класса.
- 2 Какой класс является абстрактным? При каком условии он обязан быть объявлен абстрактным?
- 3 Опишите правила использования абстрактных классов.
- 4 Дайте определение «Наследованию» в рамках ООП.
- 5 Дайте определение «Полиморфизму» в рамках ООП.
- 6 Назовите основные способы организации «Полиморфизма».
- 7 Перечислите основные преимущества «Полиморфизму» в рамках ООП
- 8 Поясните понятие интерфейса. Назначение и правила использования. Преимущества.
- 9